
UNIT9 PYTHON GUI FRAMEWORKS

Structure

- 9.0 Introduction
- 9.1 Objectives
- 9.2 TkInter
- 9.3 PyQt
- 9.4 wxPython
- 9.5 PySimpleGUI
- 9.6 Kivy
- 9.7 Summary
- 9.8 Solution to Check your Progress
- 9.9 Further Readings

9.0 INTRODUCTION

In today's age of digital transformation, software applications are increasingly built around graphical user interfaces (GUIs). Although command-line tools continue to offer robust functionality, modern users prefer applications that are intuitive, interactive, and visually appealing. Python, widely appreciated for its clarity and flexibility, has become a popular choice for developing GUIs, expanding its use beyond data science and automation into user-friendly application design.

This Unit, "Python GUI Frameworks," aims to serve as a comprehensive guide to the rich landscape of GUI libraries available in Python. Whether you are a beginner building your first software application or an experienced developer choosing the right toolkit for a complex project, this unit will guide you in exploring and mastering widely used, powerful frameworks.

In this unit, we will explore a range of popular frameworks such as Tkinter, PyQt, wxPython, PySimpleGUI and Kivy. Each framework has its own strengths, philosophies, and use cases—from lightweight tools to high-performance cross-platform applications. This unit clearly illustrates the concepts through examples. Through practical examples, detailed explanations, and comparison charts, you'll gain hands-on experience in designing responsive and user-friendly interfaces.

By the end of this unit, you will not only understand how to create GUI applications using Python but also how to choose the most suitable framework based on your project's goals, platform requirements, and development constraints.

9.1 OBJECTIVES

After completing this unit, you will be equipped to:

- Develop a basic understanding of how to design and implement GUIs using Python,

- Identify and differentiate between popular Python GUI frameworks such as Tkinter, PyQt, wxPython, PySimpleGUI and Kivy,
- Compare GUI frameworks based on features, complexity, cross-platform support, and use cases,
- Develop simple to intermediate GUI applications using selected Python frameworks, and
- Choose the most appropriate GUI framework based on project requirements and target users.

9.2 Tkinter

Tkinter is Python's built-in library for developing graphical user interfaces, making it one of the easiest and most commonly used tools for creating desktop applications. It is based on the Tcl/Tk GUI toolkit and provides a straightforward way to design interface elements like windows, buttons, menus, and dialog boxes using only Python.

Although it is simple to learn, Tkinter is powerful enough to create fully functional, responsive GUI applications. It is often recommended for beginners due to its intuitive syntax and minimal setup requirements.

Key Features of Tkinter

- **Built-in with Python:** No installation required; works out of the box.
- **Lightweight and Fast:** Suitable for small to medium-sized applications.
- **Cross-platform:** Works on Windows, macOS, and Linux.
- **Extensive Widget Set:** Includes standard UI components like labels, buttons, entries, frames, menus, and more.
- **Event-driven Programming:** Supports event loops and callbacks.
- **Canvas Widget:** Enables custom graphics and simple drawing operations.
- **Simple Layout Managers:** Includes pack(), grid(), and place() for arranging widgets.

Tkinter Widgets

In Tkinter, **widgets** are the building blocks of the interface. Everything that a user interacts with—like a button, text box, or label—is a widget. You create them by instantiating Tkinter classes and place them in a container window, usually the root window or a frame.

Below is a list of essential Tkinter widgets used in Python GUI applications:

Widget	Description
Label	Displays text or images
Button	Creates a clickable button
Entry	Accepts single-line user input
Text	Accepts multi-line text input

Widget	Description
Checkbutton	Represents a checkbox (on/off)
Radiobutton	Allows single selection among multiple options
Listbox	Displays a list of items
Scale	Creates a slider for selecting numeric values
Spinbox	Provides a fixed-range input field with increment arrows
Canvas	Supports drawing graphics and shapes
Frame	Acts as a container for grouping other widgets
Message	Displays text similar to Label, but with word wrapping
Menu	Adds menus to the window

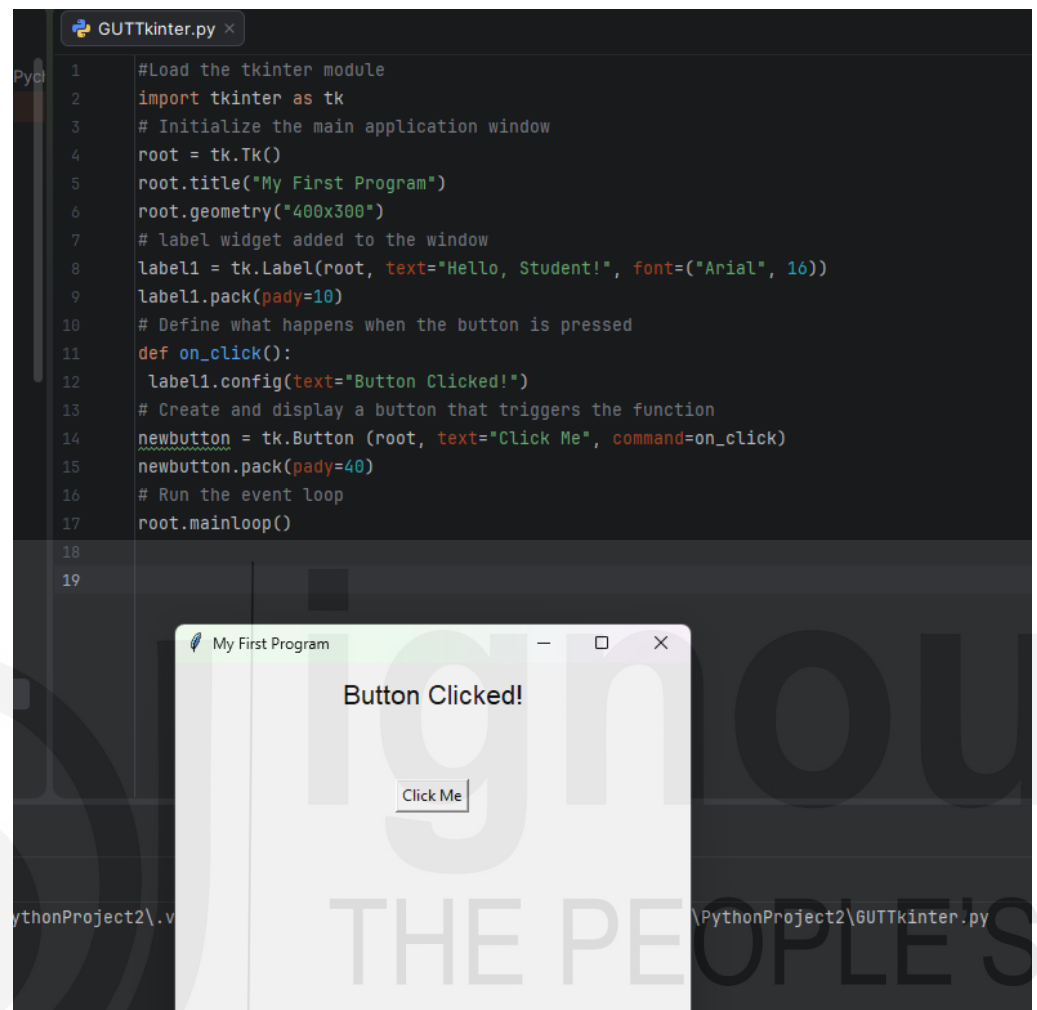
Example : Basic window featuring a label and a button using Tkinter:

```
#Load the tkinter module
import tkinter as tk
# Initialize the main application window
root = tk.Tk()
root.title("My First Program")
root.geometry("400x300")
# label widget added to the window
label1 = tk.Label(root, text="Hello, Student!", font=("Arial", 16))
label1.pack(pady=10)
# Define what happens when the button is pressed
def on_click():
    label1.config(text="Button Clicked!")
# Create and display a button that triggers the function
newbutton = tk.Button (root, text="Click Me", command=on_click)
newbutton.pack(pady=40)
# Run the event loop
root.mainloop()
```

Program Explanation

1. Load the tkinter module 'import tkinter as tk'.
2. root = tk.Tk(): Creates the main window (the root window).
3. root.title(): Sets the window title to 'My First Program'.
4. root.geometry(): Sets the window size.
5. tk.Label(): Creates a label widget with the text 'Hello, Student!' in Arial font, size 16.
6. label.pack(): Adds the label to the window with vertical padding of 10 pixels.
7. def on_click(): Defines a function that changes the label text to 'Button Clicked!' when called.
8. tk.Button(): Creates a button labeled 'Click Me'. When clicked, it calls the 'on_click' function.
9. button.pack(): Adds the button to the window with vertical padding of 10 pixels.

10. Calling `root.mainloop()` launches the main loop, which keeps the GUI running and interactive.



When you run this program:

1. A window opens titled 'My First Program'.
2. It displays a label with the text 'Hello, Student!'.
3. When you click the 'Click Me' button, the label changes to 'Button Clicked!'.

The Output of the program is shown in Figures 1 and 2.

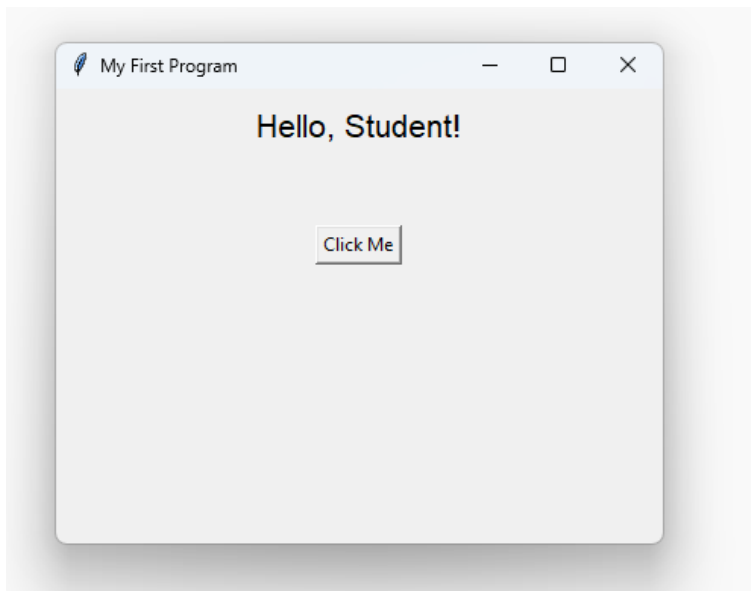


Figure 1: Output Screen for Basic Window Program

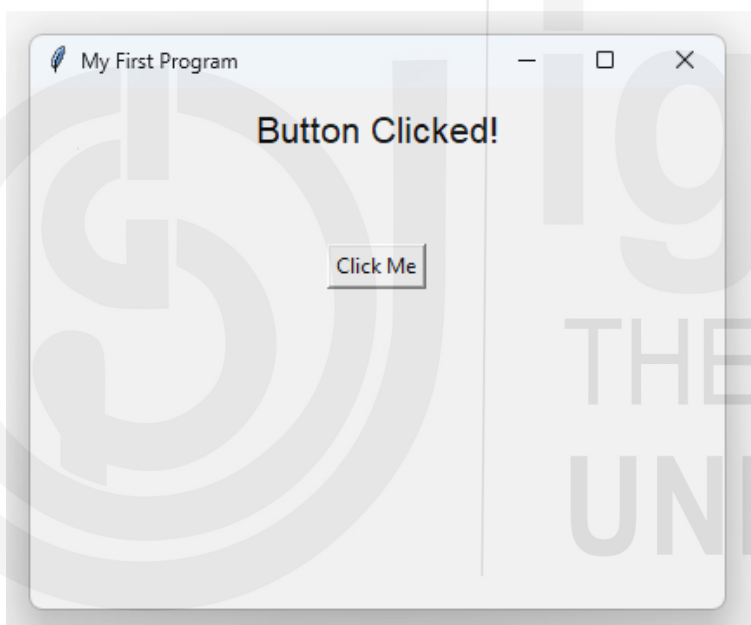


Figure 2: Output Screen after Button Clicked

Layout Management in Tkinter

To organize widgets in a window, Tkinter provides layout managers:

Layout Managers:

`pack()` – Stacks widgets vertically or horizontally.

`grid()` – Places widgets in rows and columns (like a table).

`place()` – Positions widgets at specific x/y coordinates.

Example :using grid():

```
tk.Label(root, text="Username").grid(row=0, column=0)  
tk.Entry(root).grid(row=0, column=1)
```

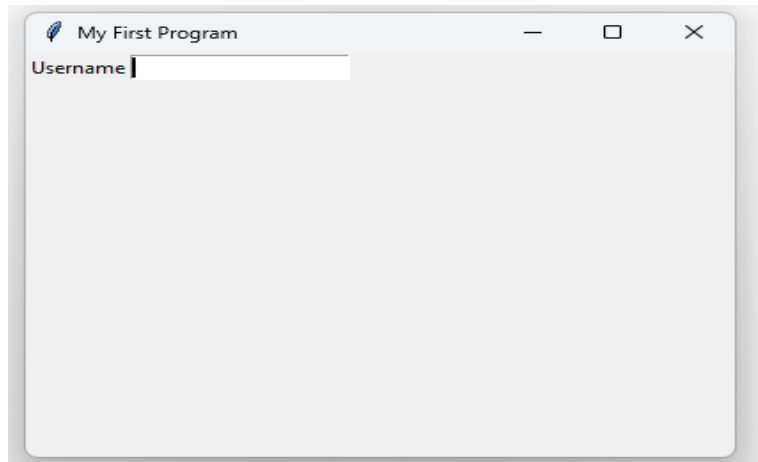


Figure 3: Output Screen for using grid():

Event Handling in Tkinter

GUIs are interactive. Tkinter allows event-driven programming through callbacks – functions that are triggered when an event (like a click) occurs.

Example: Button with Callback

```
def greet():  
    print("Hello User!")  
  
btn = tk.Button(root, text="Welcome", command=greet)  
btn.pack()
```

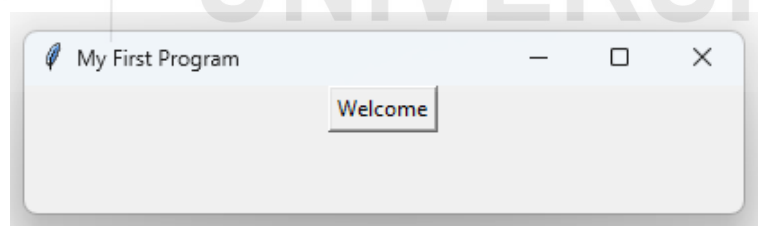


Figure 4: Output Screen for Button with Callback

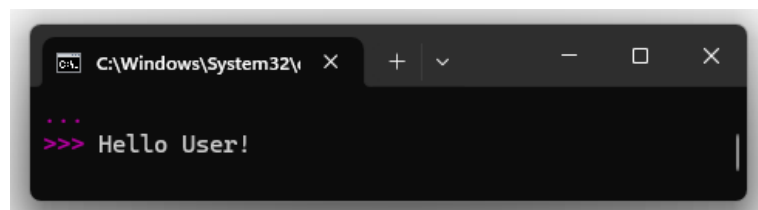


Figure 5: Output Screen after button clicked

Example: Simple Login Form

```
import tkinter as tk
def login():
    user = username.get()
    pwd = password.get()
    print(f"Username: {user}, Password: {pwd}")
root = tk.Tk()
root.title("Login Form")
tk.Label(root, text="Username").pack()
username = tk.Entry(root)
username.pack()
tk.Label(root, text="Password").pack()
password = tk.Entry(root, show='*')
password.pack()
tk.Button(root, text="Login", command=login).pack(pady=10)
root.mainloop()
```

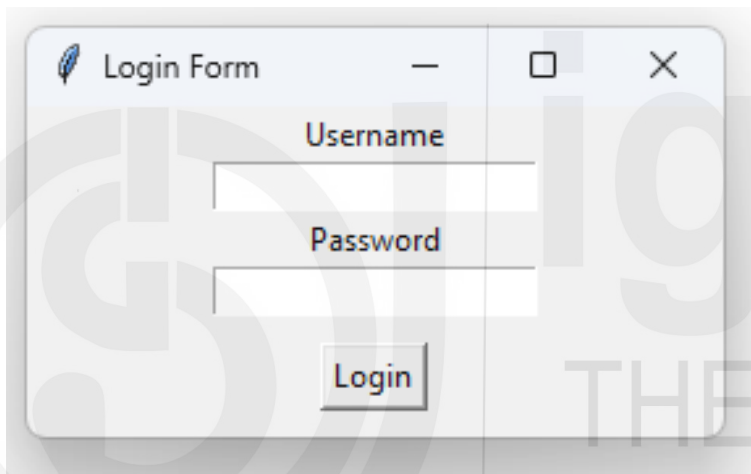


Figure 6: Output Screen for Login Form

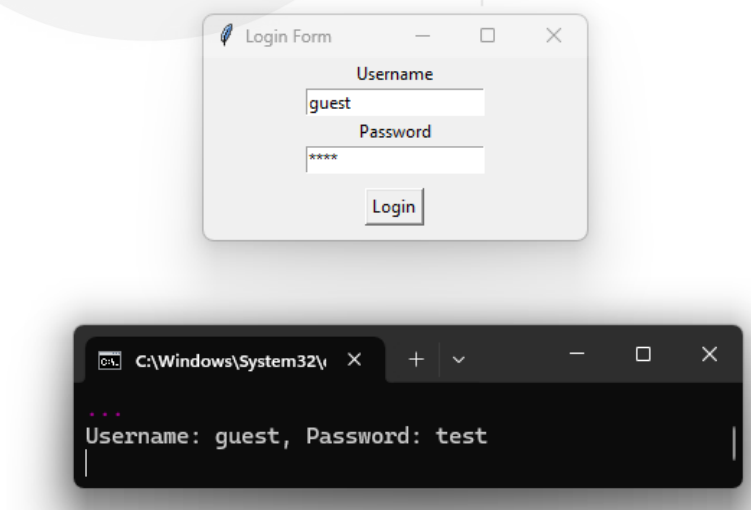


Figure 7: Output Screen for Login details filled and login button clicked

Widgets Beyond Basics

Treeview (Hierarchical Data)

Useful for displaying structured/tabular data.

```
from tkinter import ttk, Tk
root = Tk()
tree = ttk.Treeview(root, columns=("Name", "Age"), show="headings")
tree.heading("Name", text="Name")
tree.heading("Age", text="Age")
tree.insert("", "end", values=("Rohan", 24))
tree.insert("", "end", values=("Mohan", 27))
tree.pack()
root.mainloop()
```

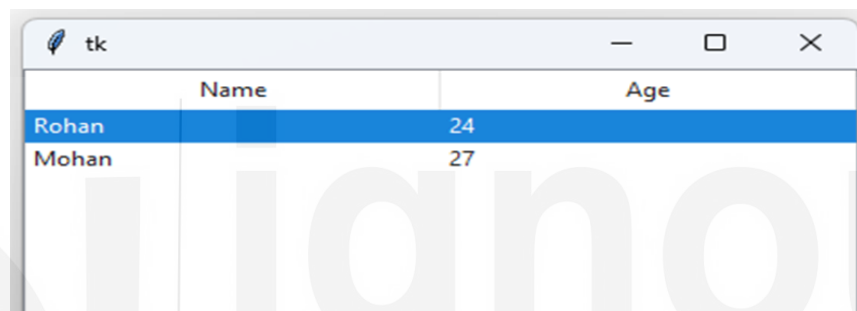


Figure 8: Output Screen for Treeview

Notebook (Tabbed Interface)

Allows multiple views within a single window.

```
from tkinter import ttk, Tk, Label
root = Tk()
notebook = ttk.Notebook(root)
frame1 = ttk.Frame(notebook)
Label(frame1, text="Home Page").pack(pady=20)
Label(frame2, text="Settings Page").pack(pady=20)
notebook.add(frame1, text="Home")
notebook.add(frame2, text="Settings")
notebook.pack(expand=True, fill="both")
root.mainloop()
```

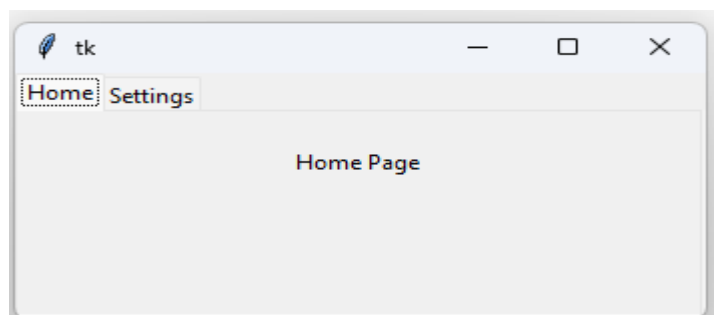


Figure 9: Output Screen for Notebook

Object-Oriented Tkinter Design

For large applications, **OOP design** is essential.
Encapsulating GUI in classes makes code modular and reusable.

```
import tkinter as tk
class MyApp(tk.Tk):
    def __init__(self):
        super().__init__()
    self.title("OOP Tkinter Example")
    self.geometry("300x200")
    tk.Button(self, text="Click Me",
        command=self.show_message).pack(pady=40)
    def show_message(self):
        tk.Label(self, text="Hello, OOP World!").pack()
app = MyApp()
app.mainloop()
```

Screenshot of execution of program:

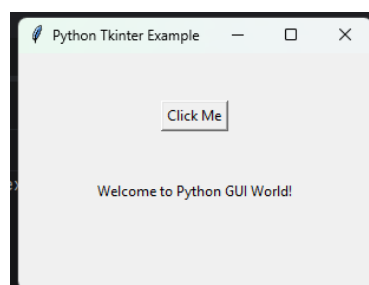
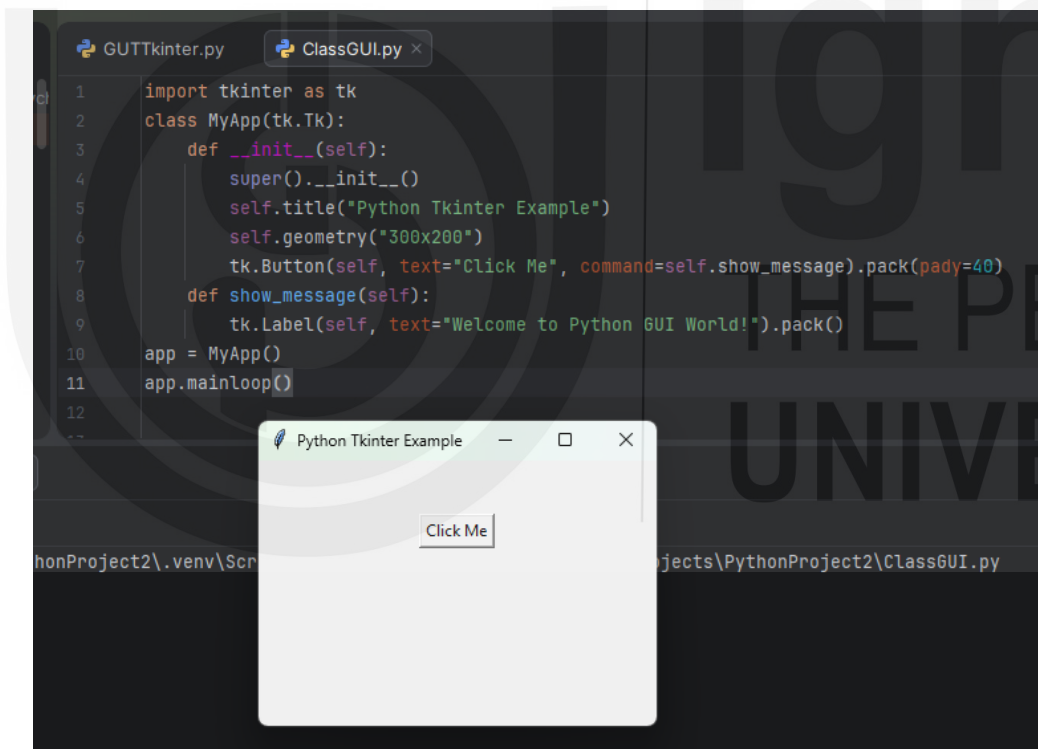


Figure 10: Output Screen for Notebook

Integrating Tkinter with Databases

Real-world applications require persistent data storage. Tkinter integrates seamlessly with **SQLite** or **MySQL**.

Example (SQLite + Tkinter):

```
import tkinter as tk
import sqlite3
def save_data():
    conn = sqlite3.connect("students.db")
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS students (name TEXT, roll
    INTEGER)")
    c.execute("INSERT INTO students VALUES (?, ?)", (entry_name.get(),
    entry_roll.get()))
    conn.commit()
    conn.close()
    entry_name.delete(0, tk.END)
    entry_roll.delete(0, tk.END)

root = tk.Tk()
root.title("Database Integration")
tk.Label(root, text="Name"). pack()
entry_name = tk.Entry(root)
entry_name.pack()
tk.Label(root, text="Roll").pack()
entry_roll = tk.Entry(root)
entry_roll.pack()
tk.Button(root, text="Save", command=save_data).pack(pady=10)
root.mainloop()
```

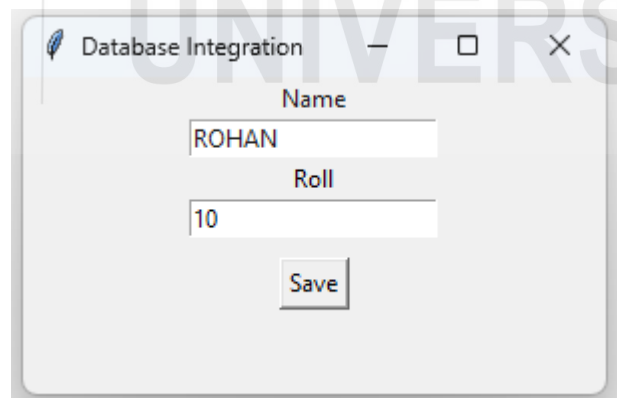


Figure 11: Output Screen for Database Integration

Threading in Tkinter

Long-running tasks (like downloads) freeze the UI if you run them in the main thread. Tkinter supports **threading** for parallelism.

```
import tkinter as tk
import threading, time
```

```
def long_task():
    for i in range(5):
        time.sleep(1)
        label.config(text=f"Step {i+1} completed")
def start_task():
    threading.Thread(target=long_task, daemon=True).start()
root = tk.Tk()
root.title("Threading Example")

label = tk.Label(root, text="Click Start", font=("Arial", 14))
label.pack(pady=20)

tk.Button(root, text="Start Task", command=start_task).pack()
root.mainloop()
```

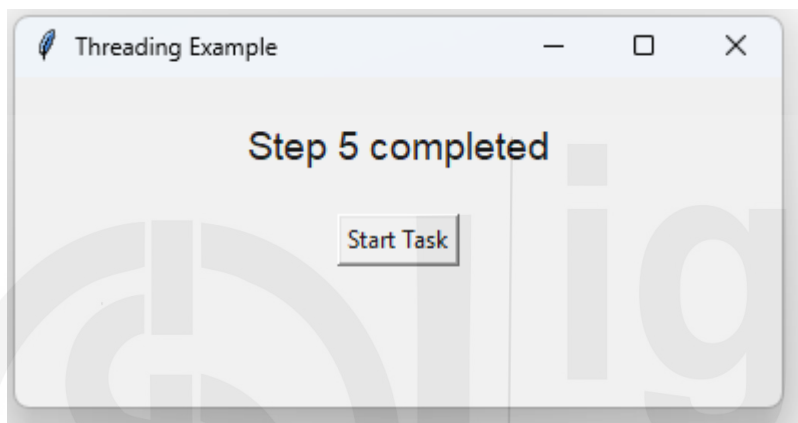


Figure 12: Output Screen for Threading

Advantages of Using Tkinter

- Easy to learn and implement, especially for beginners.
- No external dependencies since it is included with Python.
- Large community support and extensive documentation.
- Good for prototyping and building quick internal tools or utilities.

Limitations of Tkinter

- Basic visual appearance compared to modern GUI frameworks.
- Limited native look and feel, especially on macOS.
- Less suitable for complex or highly customized applications.

Check Your Progress 1

1. Which module do you import to start working with Tkinter?

- a) pygui
- b) tkinter
- c) gui_toolkit
- d) tkmodule

2. Which method is used to start the Tkinter event loop?

- a) start()
- b) launch()

- c) run()
- d) mainloop()

3. Which widget is used to display a simple piece of text?

- a) Label
- b) Button
- c) Textbox
- d) Canvas

4. What does the command attribute of a button do?

- a) Assigns a color
- b) Executes a function on click
- c) Sets the text of the button
- d) Binds a mouse event

5. The method _____ is used to place a widget inside a window using the default layout.

Q6. The _____ widget is used to take text input from the user.

Q7. The _____ widget is used to draw shapes like lines and circles.

Q8. The _____ method is used to change the text of a label at runtime.

9. Write a short code snippet to create a window with a title "Welcome".

10. What is the difference between pack(), grid(), and place() in Tkinter?

11. Write a Tkinter program to create an Entry box that accepts user input and displays it in a Label when a button is pressed.

12. Create a Student Management System in Tkinter with the following features:

- a) Add Student → Enter Name, Roll No, and Marks
- b) View Students → Display all students in a table.
- c) Delete Student → Select and delete a student.
- d) Exit → Close the application.

9.3 PyQt

PyQt is a Python library that provides bindings for the **Qt framework**, a robust and popular toolkit used to build cross-platform graphical user interfaces. With PyQt, developers can efficiently design and develop sleek, interactive, high-performance GUI applications in Python.

There are two main versions:

- **PyQt5:** Based on Qt 5, suitable for most modern applications.
- **PyQt6:** The latest version based on Qt 6, with updated APIs and features.

Key Features of Tkinter

- **Cross-platform:** Allows the application to run on Windows, Linux, and macOS without changes.
- **Rich widget set:** Includes buttons, text fields, tables, tabs, trees, graphics views, and more.
- **Customizable:** Supports custom widget creation and themes.
- **Open Source (GPL/LGPL):** Can be used freely under certain licenses.
- **Integrated with Python:** Use Python's simplicity and libraries along with Qt's power.

Installing PyQt

You can install PyQt using pip, Python's package manager. For example, to install PyQt5:

```
bash
pip install PyQt5
```

Or for PyQt6:

```
bash
pip install PyQt6
```

You may also want to install the Qt Designer (a GUI layout tool) via tools like pyqt5-tools:

```
bash
pip install pyqt5-tools
```

Basic Structure of a PyQt Application

A typical PyQt application follows these steps:

1. Import required modules
2. Create an application object
3. Create GUI components
4. Set properties and layout
5. Display the GUI
6. Run the application loop

Example: Let us write a basic PyQt6 application that creates a window with a button.

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QPushButton
# basic application created
app = QApplication(sys.argv)
# main window created
window = QWidget()
window.setWindowTitle("PyQt Simple Window")
window.setGeometry(300, 300, 300, 300)# x, y, width, height
# button created
button = QPushButton("Submit", window)
button.move(100, 100)
# display the window
window.show()
# Running of application loop
sys.exit(app.exec())
```

Explanation:

- QApplication initializes the application.
- QWidget is the base class for windows and controls.
- QPushButton is a basic button widget.
- app.exec_() starts the GUI event loop.

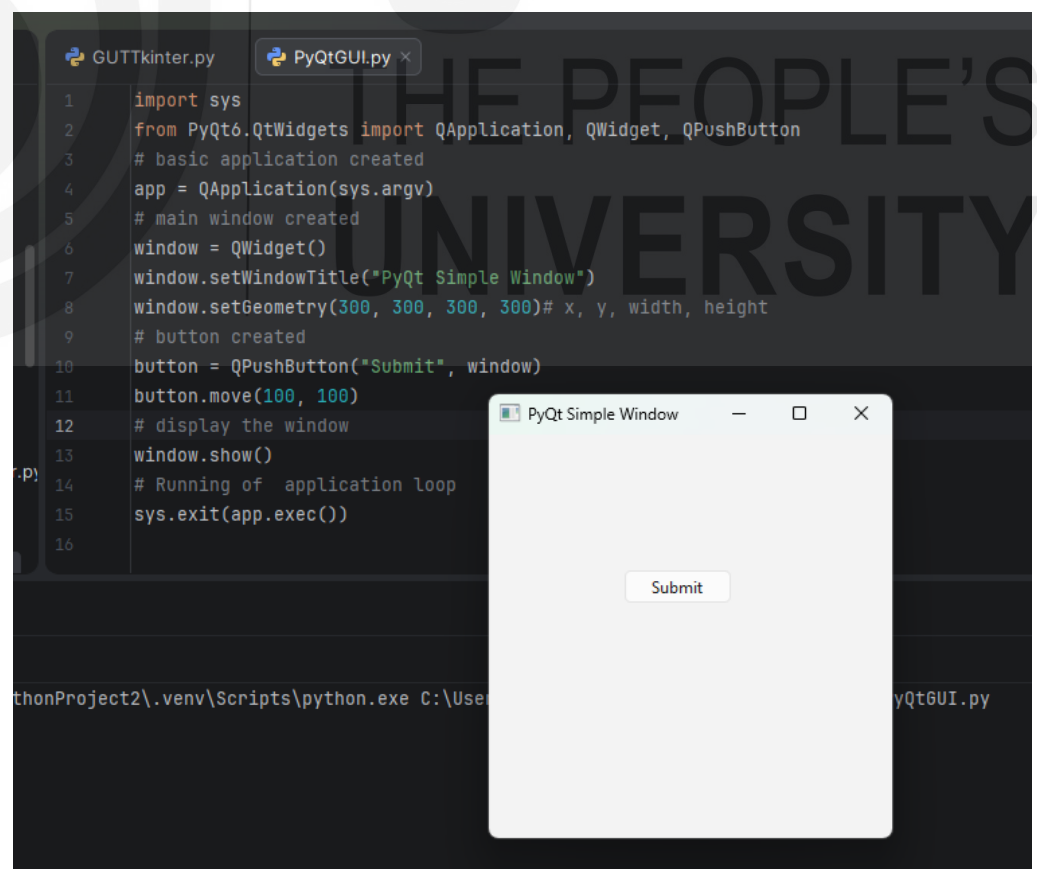


Figure 13: Output Screen for PyQt Simple Window

Common Widgets in PyQt

Widget	Description
QLabel	Display text or images
QPushButton	A clickable button
QLineEdit	Single-line text input
QTextEdit	Multi-line text editor
QCheckBox	Checkbox (true/false)
QComboBox	Dropdown menu
QListWidget	List of selectable items
QTableWidget	Table-like structure for data display

Designing with Qt Designer

Using Qt Designer, developers can build interfaces by dragging and dropping elements, then save the layout as a .ui file. To use it in Python, convert it with the command below:

```
bash
pyuic6filename.ui -o output.py
```

This speeds up development and separates design from logic.

Signals and Slots

PyQt uses **signals and slots** to handle events (like button clicks). For example:

```
button.clicked.connect(my_function)
```

Here, clicked is a signal emitted by the button, and my_function is a slot (a function) that will be called when the signal is emitted.

```
from PyQt6.QtWidgets import QApplication, QPushButton
def on_click():
    print("Button clicked!")
```

```
app = QApplication([])
btn = QPushButton("Click Me")
btn.clicked.connect(on_click)
btn.show()
app.exec()
```

clicked is a **signal** emitted when we clicked the button.
on_click is a **slot** is a function that responds to the signal.

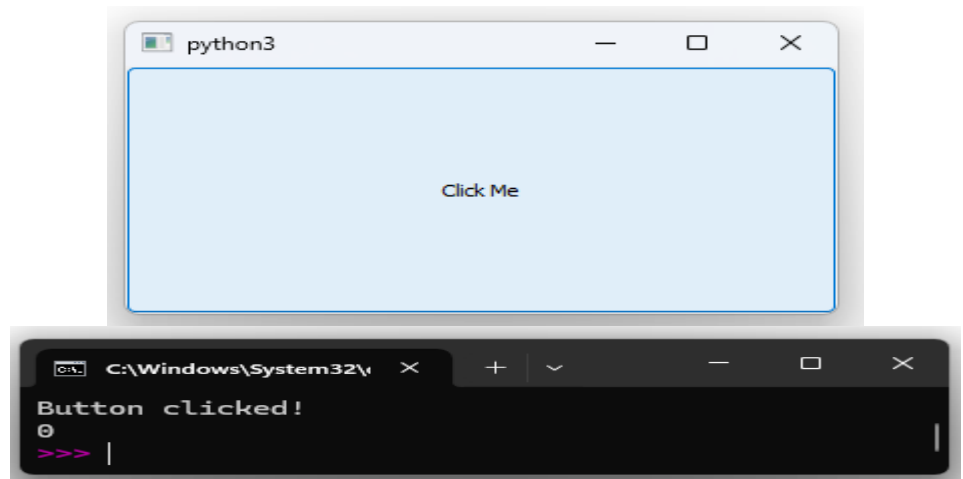


Figure 14: Output Screen for Signals and Slots

Layout Management

PyQt provides layout managers to arrange widgets:

- QVBoxLayout: Vertical layout
- QHBoxLayout: Horizontal layout
- QGridLayout: Grid layout

Example of demonstration of Layout Management:

```
from PyQt6.QtWidgets import QVBoxLayout, QPushButton, QWidget,
QApplication
app = QApplication([])
window = QWidget()
newlayout = QVBoxLayout()
newlayout.addWidget(QPushButton("One"))
newlayout.addWidget(QPushButton("Two"))
newlayout.addWidget(QPushButton("Three"))
window.setLayout(newlayout)
window.show()
app.exec()
```

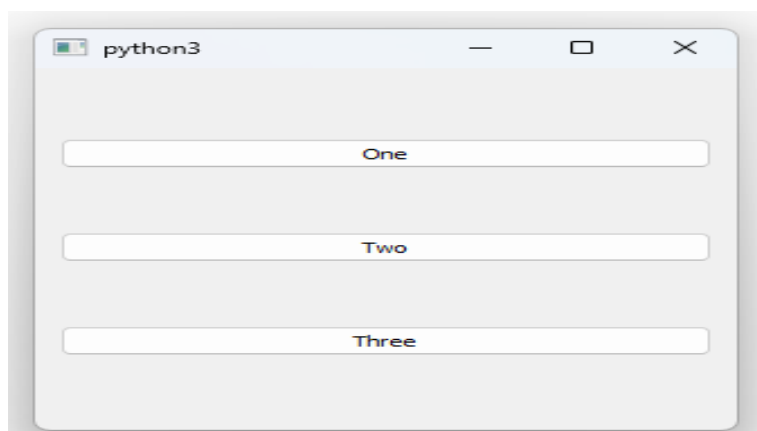


Figure 15: Output Screen for Layout Management

Model–View Programming

For handling large data sets, PyQt supports **Model–View architecture**:

- QTableWidget (simple table widget)
- QTableView + QAbstractTableModel (for flexible data handling)

This design separates data storage (model) from its presentation (view).

Multithreading

GUI applications should remain responsive. Long-running tasks (e.g., file downloads, computations) must run in background threads.

PyQt provides:

- QThread (for custom worker threads)
- QRunnable with QThreadPool (for lightweight tasks)

Persistent Storage

PyQt can store data using:

- JSON files (lightweight, portable)
- SQLite (embedded relational database)
- Pickle or external APIs

This lets you save data between sessions.

Example : Expense Tracker with Charts

```
import sys
import json
import os
from PyQt6.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QHBoxLayout,
    QLineEdit, QPushButton, QTableWidget, QTableWidgetItem,
    QComboBox, QLabel, QMessageBox
)
from PyQt6.QtCore import Qt

# Matplotlib imports for chart embedding
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
FigureCanvas
from matplotlib.figure import Figure

DATA_FILE = "expenses.json"

class ExpenseTracker(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("PyQt6 Expense Tracker with Charts")
        self.resize(800, 500)
        self.expenses = []
        self.load_expenses()

        layout = QVBoxLayout()
```

```
# --- Input Section ---
input_layout = QHBoxLayout()
self.amount_input = QLineEdit()
self.amount_input.setPlaceholderText("Amount")
self.note_input = QLineEdit()
self.note_input.setPlaceholderText("Note")
self.category_combo = QComboBox()
self.category_combo.addItem("Food", "Travel", "Shopping", "Bills",
"Other")
add_btn = QPushButton("Add Expense")
add_btn.clicked.connect(self.add_expense)

input_layout.addWidget(self.amount_input)
input_layout.addWidget(self.note_input)
input_layout.addWidget(self.category_combo)
input_layout.addWidget(add_btn)
layout.addLayout(input_layout)

# --- Filter Section ---
filter_layout = QHBoxLayout()
self.filter_combo = QComboBox()
self.filter_combo.addItem("All", "Food", "Travel", "Shopping", "Bills",
"Other")
self.filter_combo.currentIndexChanged.connect(self.refresh_table)
filter_layout.addWidget(QLabel("Filter by Category:"))
filter_layout.addWidget(self.filter_combo)
layout.addLayout(filter_layout)

# --- Expense Table ---
self.table = QTableWidgetItem(0, 3)
self.table.setHorizontalHeaderLabels(["Amount", "Category", "Note"])
self.table.horizontalHeader().setStretchLastSection(True)
layout.addWidget(self.table)

# --- Chart Section ---
self.figure = Figure(figsize=(5, 3))
self.canvas = FigureCanvas(self.figure)
layout.addWidget(self.canvas)

# --- Save Button ---
save_btn = QPushButton("Save Expenses")
save_btn.clicked.connect(self.save_expenses)
layout.addWidget(save_btn)

self.setLayout(layout)
self.refresh_table()
self.update_chart()

def load_expenses(self):
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r") as f:
            self.expenses = json.load(f)
```

```

def save_expenses(self):
    with open(DATA_FILE, "w") as f:
        json.dump(self.expenses, f, indent=2)
    QMessageBox.information(self, "Saved", "Expenses saved successfully!")

def add_expense(self):
    try:
        amount = float(self.amount_input.text().strip())
        note = self.note_input.text().strip()
        category = self.category_combo.currentText()
        if amount > 0:
            self.expenses.append({"amount": amount, "category": category, "note": note})
            self.amount_input.clear()
            self.note_input.clear()
            self.refresh_table()
            self.update_chart()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter a valid amount!")

def refresh_table(self):
    category_filter = self.filter_combo.currentText()
    filtered = self.expenses if category_filter == "All" else [
        e for e in self.expenses if e["category"] == category_filter
    ]

    self.table.setRowCount(len(filtered))
    for row, exp in enumerate(filtered):
        self.table.setItem(row, 0, QTableWidgetItem(f"{exp['amount']:.2f}"))
        self.table.setItem(row, 1, QTableWidgetItem(exp["category"]))
        self.table.setItem(row, 2, QTableWidgetItem(exp["note"]))

def update_chart(self):
    self.figure.clear()
    categories = {}
    for exp in self.expenses:
        categories[exp["category"]] = categories.get(exp["category"], 0) +
exp["amount"]

    if categories:
        ax = self.figure.add_subplot(111)
        ax.pie(categories.values(), labels=categories.keys(), autopct="%1.1f%%")
        ax.set_title("Expense Distribution")
        self.canvas.draw()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    win = ExpenseTracker()
    win.show()
    sys.exit(app.exec())

```

Execution/Output Screenshot of above program:

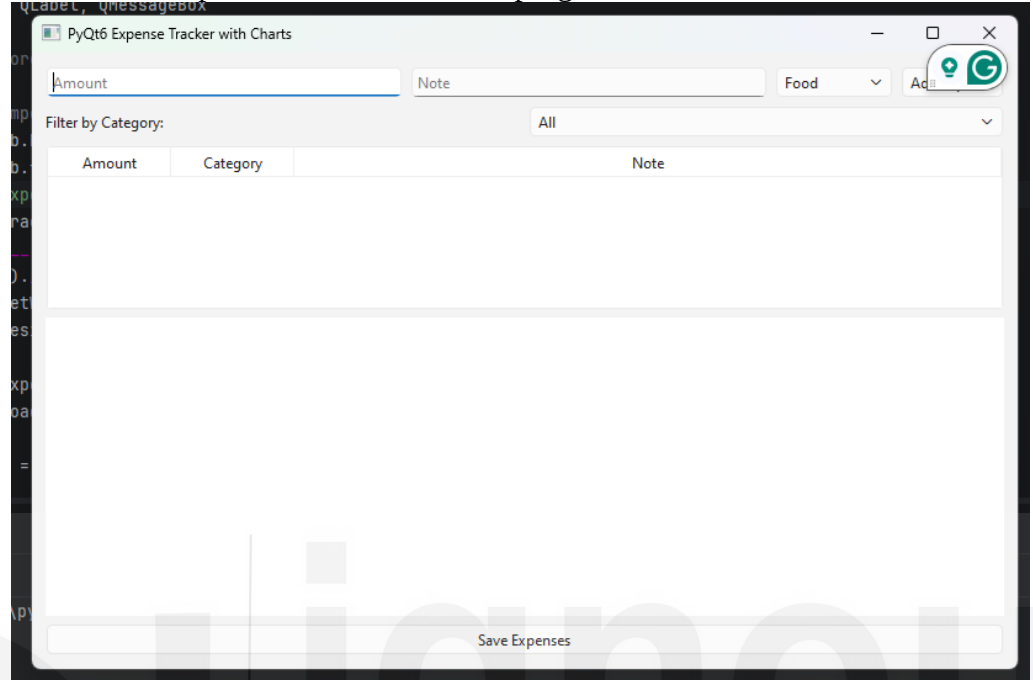


Figure 16: Execution/Output Screenshot of Expense Tracker Program

Check Your Progress 2

1. What is the purpose of QApplication in a PyQt application?
 - a) Display widgets
 - b) Handle user input
 - c) Initialize and manage the GUI application
 - d) Connect signals and slots
2. Which widget is used for creating a clickable button in PyQt?
 - a) QLabel
 - b) QPushButton
 - c) QLineEdit
 - d) QTextEdit
3. What is the method used to make a PyQt window visible?
 - a) display()
 - b) render()
 - c) show()
 - d) execute()
4. Which of the following best describes a signal in PyQt?
 - a) A visual component
 - b) A method to draw shapes
 - c) An event triggered by user interaction
 - d) A type of layout manager
5. Which command is used to convert .ui files (from Qt Designer) into Python code?

- a) qtconvert
- b) uic5py
- c) pyqtconvert
- d) pyuic5

6. What do you understand by the term “layout manager” in PyQt? Name any two types.

7. Explain the role of signals and slots in PyQt with a short example.

8. How does Qt Designer help in GUI development?

Q9. Write a PyQt application to create a simple form:

- A text input field for the Name
- A text input field for the Email
- Submit Button
- On clicking the button, print the values entered by the user to the console.

9.4 wxPython

In modern software development, building desktop applications with intuitive user interfaces is a common requirement. The wxPython is a widely used Python toolkit for building cross-platform graphical user interfaces (GUI) which resemble native applications on each operating system. It provides a Python wrapper around the wxWidgets C++ library and enables Python developers to build powerful and responsive desktop applications that look and behave like native apps on Windows, macOS, and Linux.

The wxPython is a graphical user interface (GUI) framework for Python that provides access to the wxWidgets library—a powerful, cross-platform toolkit developed in C++. With wxPython, developers can build rich, responsive desktop applications with a native appearance while writing code entirely in Python instead of C++.

Features of wxPython

- **Cross-platform:** Runs on Windows, Linux, and macOS with a native appearance.
- **Extensive Widget Library:** Provides a wide range of widgets like buttons, dialogs, panels, menus, grids, and more.
- **Native Look and Feel:** Apps look like true native apps on each platform.
- **Event-driven Programming:** Uses event handling similar to other GUI frameworks.
- **Custom Drawing Support:** Includes support for graphics, shapes, and custom widgets.

Installing wxPython

To install the latest version of wxPython, simply use pip:

```
bash
pip install wxPython
```

Basic Structure of a wxPython Program

A minimal wxPython GUI program includes:

1. Creating an application object
2. Creating a frame (main window)
3. Showing the frame
4. Starting the event loop

Example: Creating a simple Window

```
import wx
# Initialize the application without redirecting stdout/stderr
app = wx.App(False)
# Create the main application window (a frame)
frame1 = wx.Frame(None, title="Hello Student!", size=(200, 200))
# Display the frame on the screen
frame1.Show()
# application's main loop
app.MainLoop()
```

Execution/Output Screenshot:

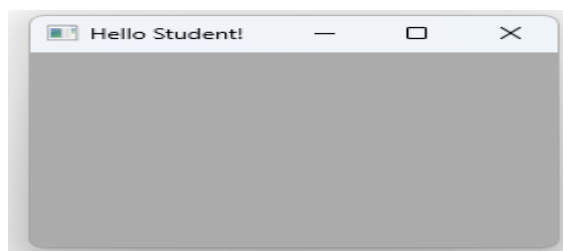


Figure 17: Output Screen for Simple Window in wxPython

Widgets (Controls)

Widgets in wxPython are known as **controls**, and they are placed inside windows (like Frame or Panel).

Example: Button and Event

```
import wx

class MyfirstFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="Simple Button")
        panel1 = wx.Panel(self)
        self.button1 = wx.Button(panel1, label="Click Me", pos=(100, 50))
        self.button1.Bind(wx.EVT_BUTTON, self.on_click)
        def on_click(self, event):
            wx.MessageBox("You have clicked the button!", "Info", wx.OK |
                wx.ICON_INFORMATION)
        app = wx.App(False)
        newframe = MyfirstFrame()
        newframe.Show()
        app.MainLoop()
```

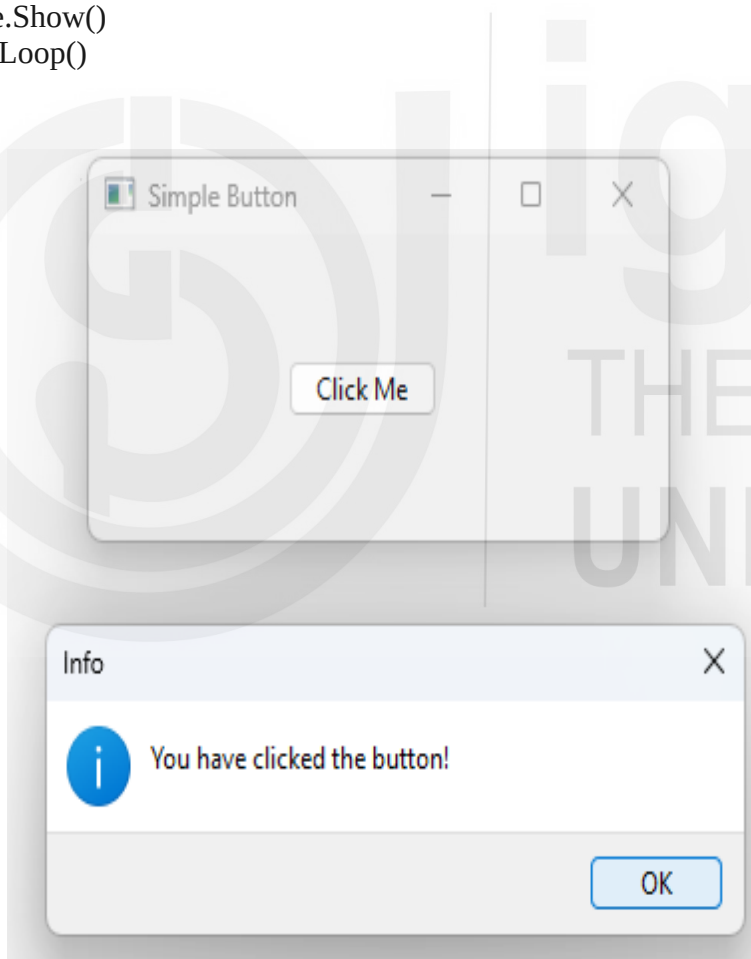


Figure 18: Output Screen for Button and Event

Common wxPython Widgets

Control	Description
wx.StaticText	Displays text
wx.TextCtrl	Text input field
wx.Button	Clickable button
wx.CheckBox	Checkbox
wx.RadioButton	Radio button
wx.ListBox	List of options
wx.ComboBox	Dropdown list
wx.Slider	Slider for selecting values
wx.Notebook	Tabbed panels

Layout Management

wxPython uses **sizers** to control widget layout.

- wx.BoxSizer: Horizontal or vertical box layout
- wx.GridSizer: Grid-like arrangement
- wx.FlexGridSizer: More flexible grid

Example: Vertical Layout using BoxSizer

```
TestFreeSimpleGUI.py  TestU9.py x
1 import wx
2 class MyFrame(wx.Frame):
3     def __init__(self, parent, title):
4         super(MyFrame, self).__init__(parent, title=title, size=(300, 200))
5         panel = wx.Panel(self)
6         sizer = wx.BoxSizer(wx.VERTICAL)
7         sizer.Add(wx.StaticText(panel, label="Student Name:"), flag=wx.ALL, border=5)
8         sizer.Add(wx.TextCtrl(panel), flag=wx.ALL, border=5)
9         panel.SetSizer(sizer)
10        self.Centre()
11        self.Show()
12
13 class MyApp(wx.App):
14     def OnInit(self):
15         frame = MyFrame(parent=None, title="Student Registration")
16         return True
17
18 if __name__ == "__main__":
19     app = MyApp()
20     app.MainLoop()
```

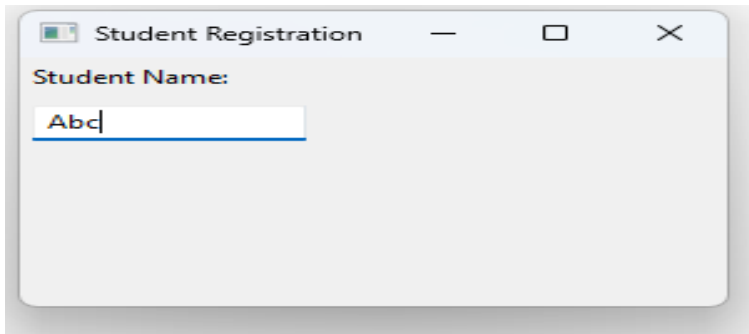


Figure 19: Output Screen for Vertical Layout

EventHandling

WxPython uses an event-driven model, where GUI elements are linked to handler functions that execute in response to user actions such as button clicks or key inputs.

Common Events:

- wx.EVT_BUTTON: When a button is clicked
- wx.EVT_TEXT: When text is entered
- wx.EVT_CLOSE: When a window is closed

Example: Button Clicked Event Handling

```
import wx
class MyApp(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="My Program")
        panel = wx.Panel(self)
        btn = wx.Button(panel, label="Submit", pos=(20, 20))
        btn.Bind(wx.EVT_BUTTON, self.on_click)
        self.Show()
        def on_click(self, event):
            wx.MessageBox("Submitted!", "Info", wx.OK)
app = wx.App(False)
frame = MyApp()
app.MainLoop()
```

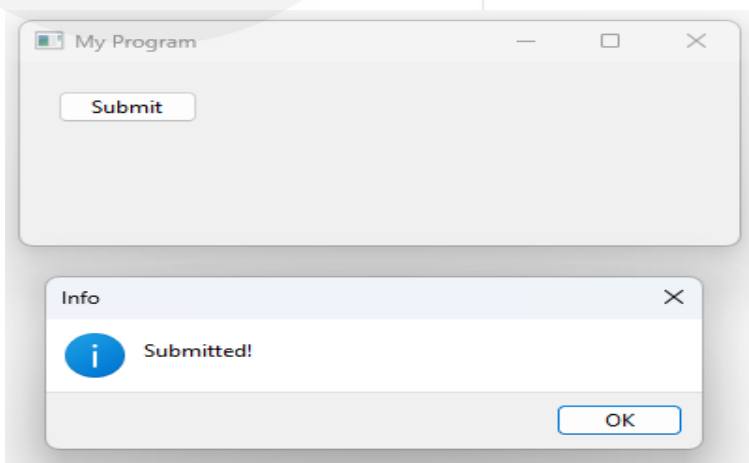


Figure 20: Output Screen for Event Handling on Button CLicked

Dialog Boxes

wxPython provides standard dialogs like:

- wx.MessageBox: Show a message
- wx.FileDialog: Open/save file dialogs
- wx.DirDialog: Select folder
- wx.ColourDialog: Color picker

```
wx.MessageBox("Operation completed", "Info", wx.OK |  
wx.ICON_INFORMATION)
```

Organizing a Multi-Panel Application

Instead of stuffing everything inside a single wx.Frame, break your app into Panels. This makes testing, reuse, and scaling easier.

```
import wx  
class InputPanel(wx.Panel):  
    def __init__(self, parent):  
        super().__init__(parent)  
        sizer = wx.BoxSizer(wx.HORIZONTAL)  
        self.txt = wx.TextCtrl(self)  
        self.btn = wx.Button(self, label="Add")  
        sizer.Add(self.txt, 1, wx.EXPAND | wx.ALL, 5)  
        sizer.Add(self.btn, 0, wx.ALL, 5)  
        self.SetSizer(sizer)
```

Here we created an InputPanel that lets the student type text and click "Add".

Displaying a Table of Data

wx.ListCtrl can be used to show multiple rows and columns.

```
class ListPanel(wx.Panel):  
    def __init__(self, parent):  
        super().__init__(parent)  
        sizer = wx.BoxSizer(wx.VERTICAL)  
        self.list = wx.ListCtrl(self, style=wx.LC_REPORT)  
        self.list.InsertColumn(0, "ID", width=60)  
        self.list.InsertColumn(1, "Task", width=250)  
        self.list.InsertColumn(2, "Status", width=100)  
        sizer.Add(self.list, 1, wx.EXPAND | wx.ALL, 5)  
        self.SetSizer(sizer)  
        def add_item(self, idx, task, status="Pending"):  
            pos = self.list.InsertItem(self.list.GetItemCount(), str(idx))  
            self.list.SetItem(pos, 1, task)  
            self.list.SetItem(pos, 2, status)
```

Putting Panels Together in a Frame

```
class MainFrame(wx.Frame):
    def __init__(self):
        super().__init__(None, title="Intermediate wxPython App", size=(600, 400))
        panel = wx.Panel(self)
        vbox = wx.BoxSizer(wx.VERTICAL)
        self.input_panel = InputPanel(panel)
        self.list_panel = ListPanel(panel)

        vbox.Add(self.input_panel, 0, wx.EXPAND)
        vbox.Add(self.list_panel, 1, wx.EXPAND)

        panel.SetSizer(vbox)
        # Event binding
        self.input_panel.btn.Bind(wx.EVT_BUTTON, self.on_add)
        self.statusbar = self.CreateStatusBar()
        self.counter = 1
```

```
    def on_add(self, event):
        text = self.input_panel.txt.GetValue().strip()
        if text:
            self.list_panel.add_item(self.counter, text)
            self.input_panel.txt.Clear()
            self.statusbar.SetStatusText(f"Added item #{self.counter}")
            self.counter += 1
```

Menus and Toolbars

```
def _build_menu(self):
    menubar = wx.MenuBar()
    file_menu = wx.Menu()
    save_item = file_menu.Append(wx.ID_SAVE, "&Save\tCtrl+S")
    load_item = file_menu.Append(wx.ID_OPEN, "&Load\tCtrl+O")
    quit_item = file_menu.Append(wx.ID_EXIT, "E&xit\tCtrl+Q")
    menubar.Append(file_menu, "&File")
    self.SetMenuBar(menubar)
    self.Bind(wx.EVT_MENU, self.on_save, save_item)
    self.Bind(wx.EVT_MENU, self.on_load, load_item)
    self.Bind(wx.EVT_MENU, lambda e: self.Close(), quit_item)
```

Dialogs (File Save / Open)

```
import json
def on_save(self, event):
    with wx.FileDialog(self, "Save tasks", wildcard="JSON files
(*.json)|*.json",
                        style=wx.FD_SAVE | wx.FD_OVERWRITE_PROMPT) as
dlg:
    if dlg.ShowModal() == wx.ID_OK:
        path = dlg.GetPath()
        data = []
        for i in range(self.list_panel.list.GetItemCount()):
            row = {
```

```
        "id": self.list_panel.list.GetItemText(i, 0),
        "task": self.list_panel.list.GetItemText(i, 1),
        "status": self.list_panel.list.GetItemText(i, 2),
    }
    data.append(row)
    with open(path, "w") as f:
        json.dump(data, f, indent=2)
    self.statusbar.SetStatusText(f"Saved {len(data)} tasks")

    def on_load(self, event):
        with wx.FileDialog(self, "Load tasks", wildcard="JSON files
(*.json)|*.json",
                           style=wx.FD_OPEN | wx.FD_FILE_MUST_EXIST) as dlg:
            if dlg.ShowModal() == wx.ID_OK:
                path = dlg.GetPath()
                with open(path) as f:
                    data = json.load(f)
        self.list_panel.list.DeleteAllItems()
        for row in data:
            self.list_panel.add_item(row["id"], row["task"], row["status"])
        self.statusbar.SetStatusText(f"Loaded {len(data)} tasks")
```

Timers for Periodic Updates

Timers can refresh the UI, update clocks, or auto-save.

```
    def start_timer(self):
        self.timer = wx.Timer(self)
        self.Bind(wx.EVT_TIMER, self.on_tick, self.timer)
        self.timer.Start(1000) # every 1 second

    def on_tick(self, event):
        self.statusbar.SetStatusText(f"Clock:
{wx.DateTime.Now().FormatISOTime()}")
```

Main Application Bootstrap

```
class MyApp(wx.App):
    def OnInit(self):
        frame = MainFrame()
        frame._build_menu()
        frame.start_timer()
        frame.Show()
        return True

if __name__ == "__main__":
    app = MyApp(False)
    app.MainLoop()
```

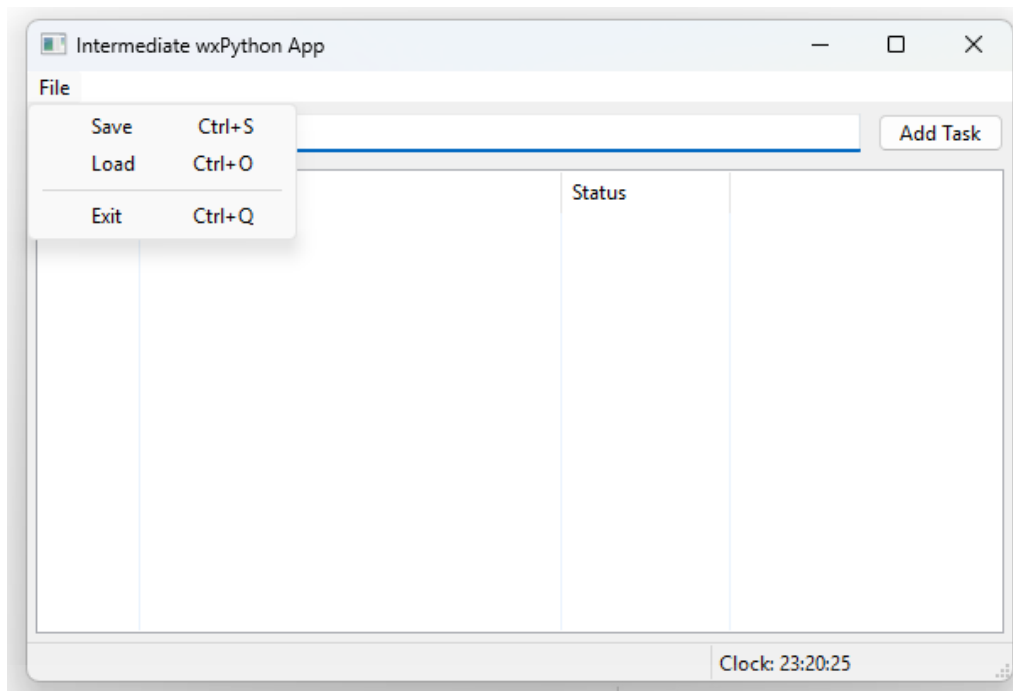


Figure 21: Output Screen for Multi-Panel Application

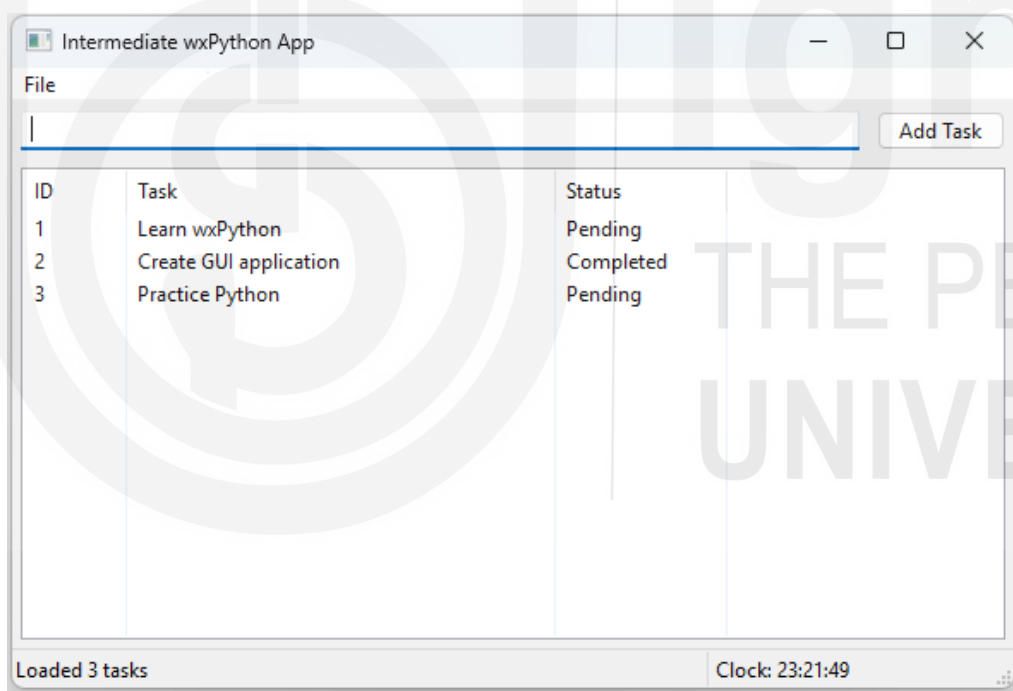


Figure 22: Output Screen for Multi-Panel Application after loading json file

Check Your Progress -3

1. Which class is used to create the main application window in wxPython?

- a) wx.Dialog
- b) wx.Panel
- c) wx.Frame
- d) wx.Window

2.What role does wx.App play in a wxPython program?

- a) Create menu items
- b) Handle events
- c) Initializing the window and entering the main loop
- d) Draw shapes on canvas

3.Which widget is used to display static text in wxPython?

- a) wx.Label
- b) wx.TextCtrl
- c) wx.StaticText
- d) wx.Message

4.What is the function of Show() in wxPython?

- a) Ends the application
- b) Prints values to console
- c) Displays a widget or window on screen
- d) Closes the frame

5.Which layout manager arranges the widgets in a column or row?

- a) wx.BoxSizer
- b) wx.GridBagSizer
- c) wx.FlexSizer
- d) wx.SimpleSizer

.....

6.Differentiate between wx.TextCtrl and wx.StaticText.

.....

7.How does a sizer help in designing GUI layouts?

.....

8. Create a Task Manager with Search, Sorting & Status Update in wxPythonwith the following features:

- Add tasks with description and priority.
- Mark tasks as *Pending*, *In Progress*, or *Completed*.
- Search tasks dynamically.
- Sort tasks by priority or status.
- Save/load tasks in JSON for persistence.
- Uses wx.Frame, wx.Panel, wx.BoxSizer, wx.ListCtrl, wx.TextCtrl, wx.ComboBox.

.....

9.5 PySimpleGUI

PySimpleGUI is a Python module that makes building graphical user interfaces easier and more accessible. It serves as a simplified layer over other popular GUI toolkits like Tkinter, Qt, wxPython, and Remi. By reducing the amount of code needed, it enables developers—especially beginners—to create user-friendly interfaces quickly. It is particularly useful for rapid prototyping and small-scale applications.

Key Features:

- Easy-to-understand syntax compared to Tkinter or PyQt
- Designed with a minimal learning curve
- Ideal for rapid development and prototyping
- Cross-platform compatibility
- Built-in support for themes and visual customization

Installing PySimpleGUI

To install PySimpleGUI, use pip:
pip install PySimpleGUI

Example of a basic GUI Application using PySimpleGUI:

```
import PySimpleGUI as sg
layout1 = [
    [sg.Text("Enter your first name:"), sg.InputText()],
    [sg.Button("OK"), sg.Button("Cancel")]
]
window = sg.Window("Basic GUI", layout1)
while True:
    event, values = window.read()
    if event == sg.WINDOW_CLOSED or event == "Cancel":
        break
    print("Hello Student!", values[0])
window.close()
```

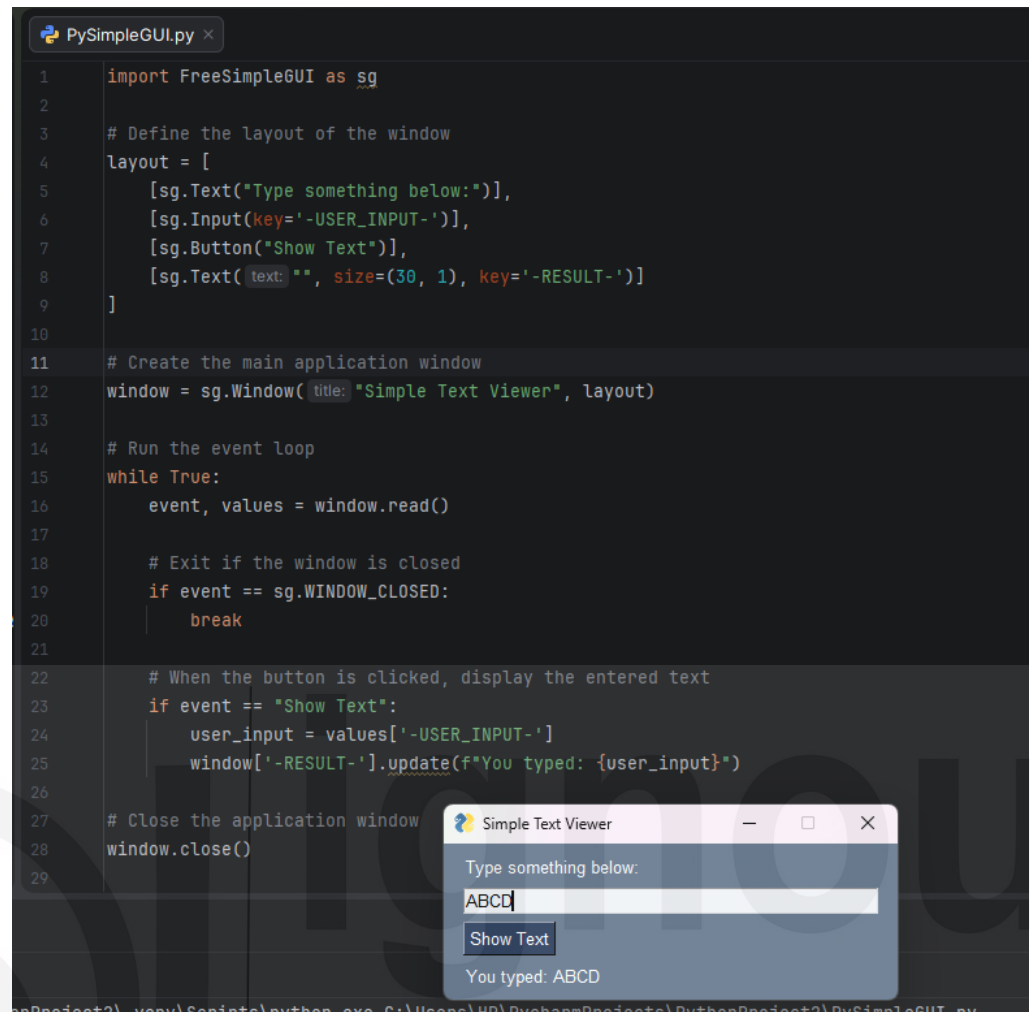


Figure 23: Output Screen for Basic GUI

Note:

If you see the screenshot given in figure 21, FreeSimpleGUI is used instead of PySimpleGUI. If you face any difficulty in running the code using PySimpleGUI as sg, you should install FreeSimpleGUI using:

python.exe -m pip install FreeSimpleGUI, then use FreeSimpleGUI as sg

Understanding Layout:

In PySimpleGUI, the interface layout is organized using a list of lists, where each inner list defines a single row of widgets. This row-wise, grid-style arrangement makes it easy to position and align elements clearly within the window.

Commonly Used Widgets:

- sg.Text(): Display static text
- sg.InputText(): Accept user input
- sg.Button(): Create a button
- sg.Checkbox(), sg.Radio(), sg.Slider(), sg.Combo(), sg.Listbox()
- sg.Image(), sg.ProgressBar(), sg.Multiline()

Event Loop:

PySimpleGUI applications use an event loop to handle user interactions. The loop runs continuously, waiting for events (e.g., button clicks), and processes them accordingly.

Customization and Themes:

PySimpleGUI offers built-in themes for styling applications. To set a theme:

```
sg.theme("DarkBlue")
```

Use 'sg.theme_previewer()' to preview all available themes.

Message Boxes and Popups:

- sg.popup(): Display information or alerts
- sg.popup_yes_no(), sg.popup_ok_cancel(), sg.popup_error(), etc.

Advantages of PySimpleGUI

- Easy to learn and use
- Excellent documentation
- Quick GUI creation with less code
- Cross-platform compatibility

Limitations of PySimpleGUI

- Less flexible than direct frameworks like PyQt or Tkinter
- Limited control over low-level widget behavior
- Advanced features may require switching to the underlying GUI framework

Non-blocking concurrency patterns:

PySimpleGUI's event loop should not be blocked. Use threads, executors, or asyncio with care.

Worker thread example (ThreadPool + Queue):

This pattern lets background tasks post updates back to the UI via a thread-safe queue polled in the event loop.

```
import threading, queue, time
from concurrent.futures import ThreadPoolExecutor
```

```
class UIQueue:
    def __init__(self):
        self.q = queue.Queue()
    def put(self, item): self.q.put(item)
    def get_nowait(self):
        try: return self.q.get_nowait()
        except queue.Empty: return None
```

```
class WorkerPool:
    def __init__(self, ui_queue, max_workers=4):
        self.uiq = ui_queue
```

```
self.pool = ThreadPoolExecutor(max_workers=max_workers)
```

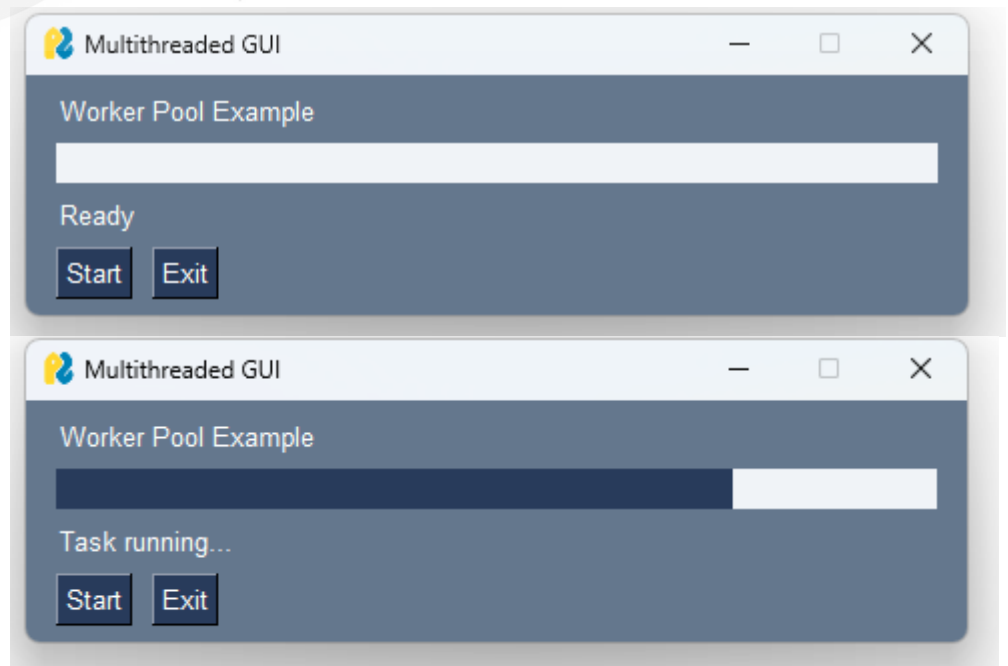
```
def submit(self, fn, *args, **kwargs):
    def wrapper():
        try:
            for progress in fn(*args, **kwargs):
                self.uiq.put(("progress", progress))
        except Exception as e:
            self.uiq.put(("error", str(e)))
    self.pool.submit(wrapper)
```

fn should be a generator that yields progress updates, or a function you adapt to report progress.

Event loop integration:

Poll the UI queue in `window.read(timeout=...)`:

```
uiq = UIQueue()
pool = WorkerPool(uiq)
while True:
    event, values = window.read(timeout=200) # 200ms
    if event == sg.WINDOW_CLOSED: break
    # handling the GUI events...
    msg = uiq.get_nowait()
    while msg:
        typ, payload = msg
        if typ == "progress":
            window["-PROG-"].update(payload)
        elif typ == "done":
            window["-STATUS-"].update("Task finished")
        elif typ == "error":
            sg.popup_error(f"Worker error: {payload}")
        msg = uiq.get_nowait()
```



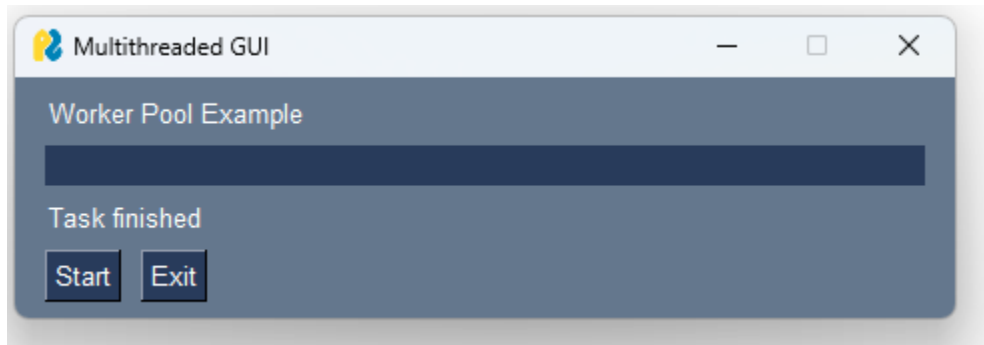


Figure 24: Output Screen for Worker thread

Asyncio Integration:

PySimpleGUI is not asyncio-native; to use asyncio, run an event loop in another thread and communicate via a queue, or use `asyncio.run_coroutine_threadsafe`. Prefer the worker queue approach for simplicity and cross-platform reliability.

```
import asyncio
import threading
import queue
import PySimpleGUI as sg

# A thread-safe queue is used to send messages from the
# background worker thread to the PySimpleGUI main thread.
ui_queue = queue.Queue()

def async_worker():
    # This function runs in a separate worker thread.
    # It creates and runs the asyncio event loop.
    async def task():
        # Simulate an asynchronous operation.
        # In a real application, this could be an HTTP request,
        # file operation, database operation, or another async task.
        for i in range(1, 6):
            await asyncio.sleep(1)

        # Send progress information to the GUI thread.
        # The worker does not directly modify GUI elements.
        ui_queue.put(("progress", i * 20))

    # Notify the GUI that the asynchronous operation is complete.
    ui_queue.put(("done", "Task completed"))

    # Start the asyncio event loop and execute the coroutine.
    asyncio.run(task())

# -----
# PySimpleGUI user interface
# -----

# The GUI contains a progress bar and buttons for
# starting the asynchronous operation or closing the application.
```

```
layout = [  
    [sg.Text("Asyncio Integration")],  
    [  
        sg.ProgressBar(  
            100,  
            orientation="h",  
            size=(30, 20),  
            key="-PROGRESS-"  
        )  
    ],  
    [  
        sg.Button("Start"),  
        sg.Button("Exit")  
    ]  
]  
  
# Create the PySimpleGUI window.  
window = sg.Window("Asyncio Integration", layout)  
  
# -----  
# Main GUI event loop  
# -----  
  
while True:  
    # timeout allows the GUI to remain responsive while  
    # periodically checking the queue for worker messages.  
    event, values = window.read(timeout=100)  
  
    # Handle window closing or Exit button.  
    if event in (sg.WINDOW_CLOSED, "Exit"):  
        break  
  
    # Start the worker thread when the Start button is pressed.  
    if event == "Start":  
        threading.Thread(  
            target=async_worker,  
            daemon=True  
        ).start()  
  
    # -----  
    # Process messages received from the worker thread.  
    # -----  
    try:  
        while True:  
            # Retrieve a message without blocking the GUI.  
            message, data = ui_queue.get_nowait()  
  
            # Update the progress bar from the GUI thread.  
            if message == "progress":  
                window["-PROGRESS-"].update(data)  
  
            # Display a completion message when the  
            # asynchronous operation has finished.
```

```
elif message == "done":
    sg.popup(data)
```

```
# The queue is empty; continue running the GUI event loop.
except queue.Empty:
    pass
```

```
# Close the PySimpleGUI window and release GUI resources.
window.close()
```

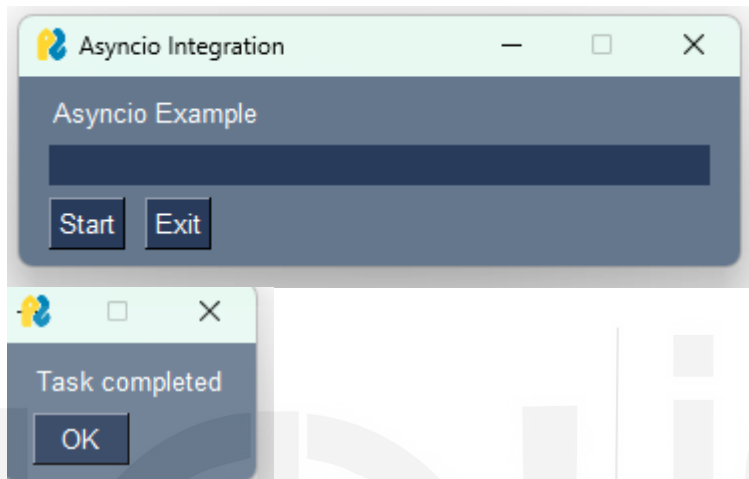


Figure 25: Output Screen for Asyncio Integration

WebSocket / push updates:

Use a background thread with websocket-client (or websockets in asyncio) and enqueue received messages to UI queue.

Theming and custom elements:

Code to register a custom theme:

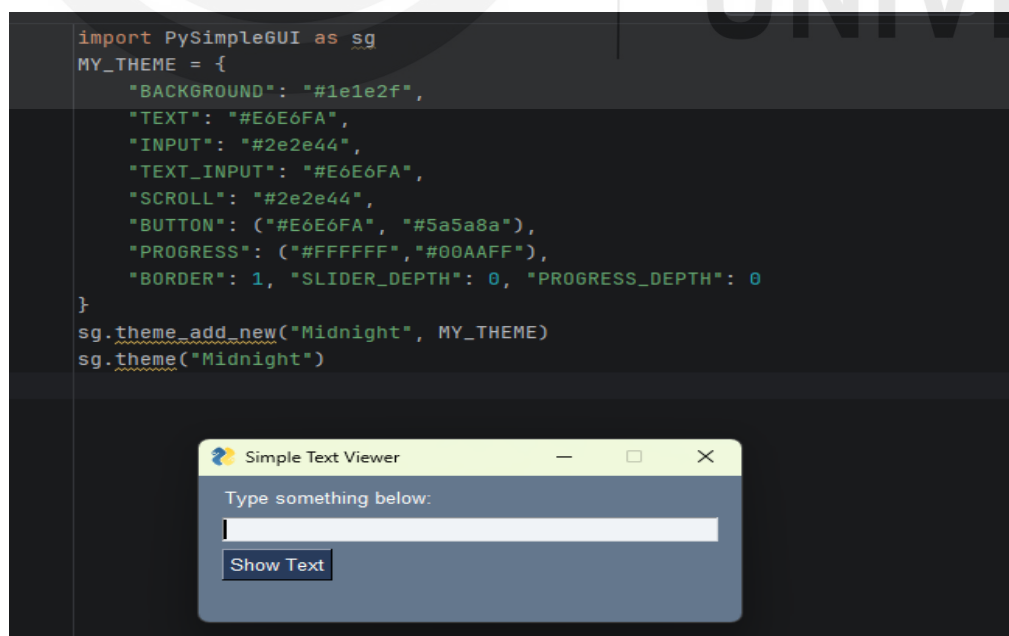


Figure 26: Output Screen for Team Update

PySimpleGUI allows theme registration (colors, fonts, element styles).

Add fonts and sizes in your layout elements, but avoid pixel-locking — prefer flexible sizes.

Custom composite element:

Create reusable UI pieces (e.g., labeled input with validation):

```
def LabeledInput(key, label, width=30):  
  
    return [sg.Text(label), sg.Input(key=key, size=(width,1))]
```

You can also encapsulate multi-element widgets into functions that return layout fragments.

Some Advanced UI elements and interactions:

Virtualized long tables:

Large tables should not update row-by-row in real time. Batch updates and use `window["-TABLE-"].update(values=rows)` with sliced subsets or pagination.

Editable tables:

PySimpleGUI Table is not fully editable; to edit, open an edit dialog on double-click and update the underlying list.

```
# on double-click  
selected = values["-TABLE-"]  
if selected:  
    row_idx = selected[0]  
    row = rows[row_idx]  
    new_val = sg.popup_get_text("Edit Task", default_text=row[1])  
    if new_val is not None:  
        rows[row_idx][1] = new_val  
        window["-TABLE-"].update(values=rows)
```

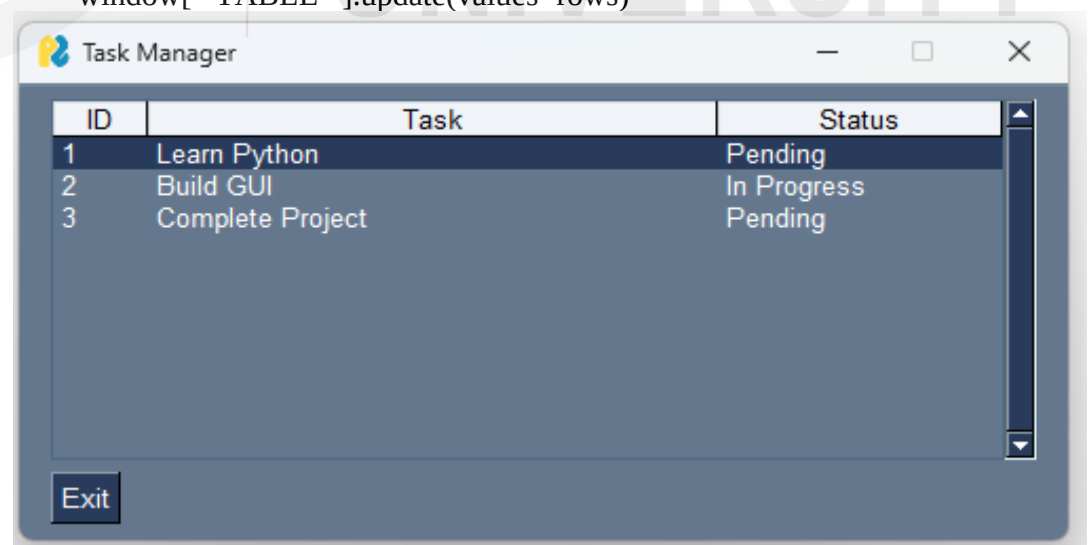


Figure 27(a): Output Screen for Editable tables

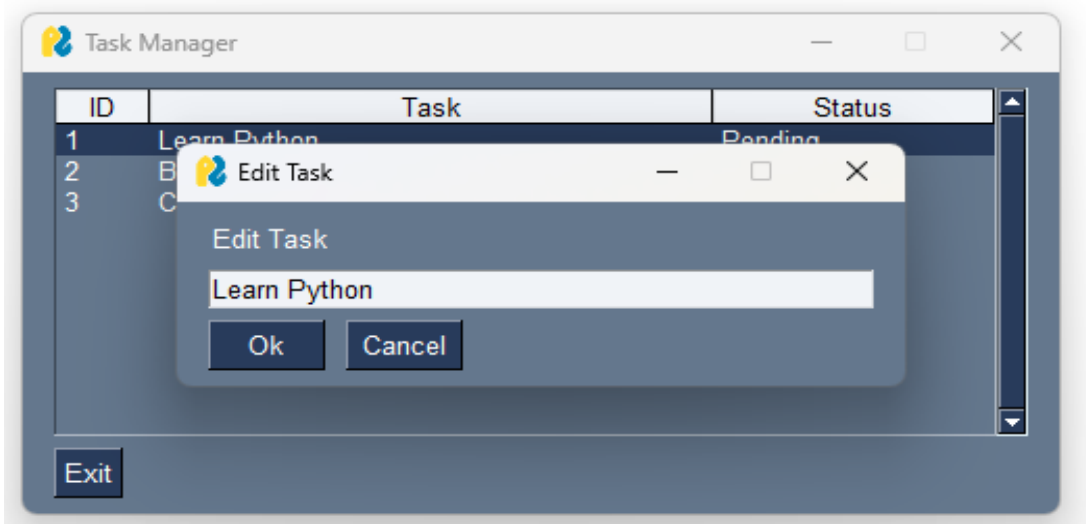


Figure 27(b): Output Screen for Editable tables

Drag & drop files:

`sg.popup_get_file(..., multiple_files=True)` is simple; for OS drag & drop, some PySimpleGUI backends support filenames appearing in values when dropping onto the window—handle event entries for that key.

Persistence & storage:

JSON and CSV

Use `json` for structured save and `csv` for interoperable spreadsheets. Keep a small adapter in `data/storage.py` that validates and sanitizes input.

SQLite for larger needs:

Use `sqlite3` for indexed storage. Wrap DB I/O in services and use background threads for heavy queries.

Plugins and extensibility:

Simple plugin loader

Allow Python modules under `plugins/` to register menu items or callbacks.

```
# plugins/__init__.py
import pkgutil, importlib
```

```
def load_plugins(app_context):
    for finder, name, ispkg in pkgutil.iter_modules(__path__):
        mod = importlib.import_module(f"{__name__}.{name}")
        if hasattr(mod, "register"):
            try:
                mod.register(app_context)
            except Exception as e:
                print("Plugin load failed:", name, e)
```

A plugin's `register(app_context)` receives references to `ui`, `services`, and `config`.

Accessibility and UX:

- Provide keyboard shortcuts (e.g., Accelerator keys in menu).
- Use clear labels, tooltips (element.Tooltip), and consistent tab order.
- Avoid tiny clickable areas — set reasonable element sizes.
- Support high-contrast themes and respect system font scaling when possible.

Check Your Progress 4

1.What is PySimpleGUI primarily designed for?

- a) Web development
- b) Game development
- c) Simplifying GUI development in Python
- d) Scientific computing

2.Which statement is used to display a window in PySimpleGUI?

- a) window.show()
- b) window.start()
- c) window.open()
- d) window.read()

3.What does the `sg.Text()` element do in PySimpleGUI?

- a) Accepts user input
- b) Displays a clickable button
- c) Shows static text
- d) Creates a new window

4.How do you close a PySimpleGUI window?

- a) window.end()
- b) window.stop()
- c) window.close()
- d) window.exit()

5.What is PySimpleGUI and why is it popular among Python developers?

.....

6.How do you create a simple input box using PySimpleGUI?

.....

7.Differentiate between sg.Button() and sg.Text().

.....

8. What is the purpose of the event loop in PySimpleGUI?

.....

.....

9. Create a program using *FreeSimpleGUI* as sg that includes an input field as text and a button. Upon clicking the button, the input text should appear beneath it.

.....

.....

9.6 Kivy

Kivy is a free and open-source Python framework used for building multi-touch and gesture-based applications. Designed to run across platforms—including Windows, Linux, macOS, Android, and iOS—it is well-suited for apps that need custom interfaces and interactive features.

Key Highlights:

Works seamlessly on multiple platforms; offers a wide variety of user interface widgets:

- Uses GPU acceleration via OpenGL for smooth graphics
- Built-in support for touch gestures and multi-touch input
- Enables declarative UI development with its KV language

Installing Kivy

To install Kivy framework, use pip:
pip install kivy

Example of developing basic Kivy application

```
from kivy.app import NewApp
from kivy.uix.label import Label
class MyApp(NewApp):
    def build(self):
        return Label(text="Hello, Student!")
if __name__ == "__main__":
    MyApp().run()
```

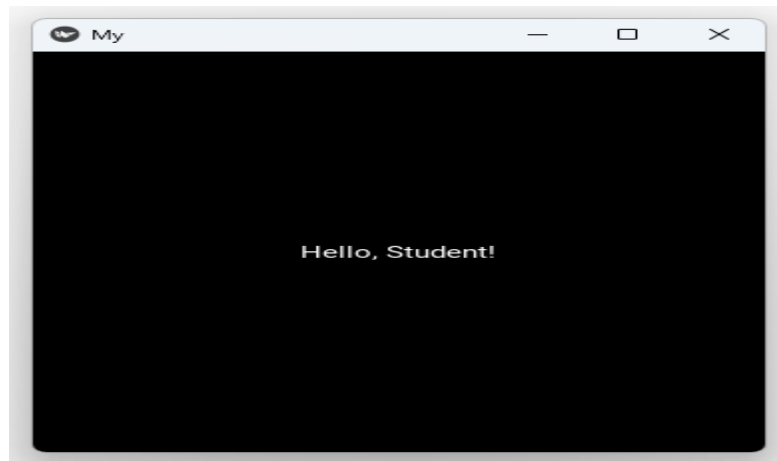


Figure 28: Output Screen for basic Kivy application

Widgets in Kivy:

Kivy provides many widgets like:

- Label
- Button
- TextInput
- BoxLayout
- GridLayout
- Image
- Slider, Switch, Checkbox, ProgressBar

Layouts:

Kivy layouts help organize widgets:

Layouts	Description
BoxLayout	Arranges widgets in rows/columns
GridLayout	Grid-like arrangement
FloatLayout	Uses absolute positioning
AnchorLayout	Positions widgets using anchor points

KV Language:

KV language is a domain-specific language used by Kivy to separate logic and presentation. It allows developers to define the UI in a clean and declarative way.

Event Handling:

Kivy widgets support event binding for interaction:

```
btn1 = Button(text="Submit")  
btn1.bind(on_press=self.callback_function)
```

The App Life Cycle:

- build(): Main method where the UI is constructed
- on_start(), on_stop(), on_pause(), on_resume(): App lifecycle hooks

Advantages of Kivy:

- Write once, run anywhere
- Open source and actively maintained
- Suited for multitouch and gesture-based applications
- Supports hardware-accelerated rendering

Limitations of Kivy:

- Larger App Size: Applications built with Kivy tend to have bigger file sizes, especially when packaged for mobile platforms.
- Not Optimal for Traditional Desktop Interfaces: Kivy is more suited for custom and touch-friendly apps, and may not blend well with standard desktop UI conventions.
- Fewer Native Widgets: Compared to native GUI toolkits, Kivy provides fewer built-in widgets that mimic the look and feel of the operating system.

Organizing Apps with Multiple Screens

Kivy apps often need more than one screen. For that, we use **ScreenManager**.

```
from kivy.app import App
from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.lang import Builder
kv = """
ScreenManager:
    MenuScreen:
    TaskScreen:
<MenuScreen>:
    name: "menu"
    BoxLayout:
        orientation: "vertical"
        Label:
            text: "Welcome to Kivy App"
        Button:
            text: "Go to Tasks"
            on_press: root.manager.current = "tasks"
<TaskScreen>:
    name: "tasks"
    BoxLayout:
        orientation: "vertical"
        Label:
            text: "Task Manager"
        Button:
            text: "Back"
            on_press: root.manager.current = "menu"
"""
```

```
class MenuScreen(Screen):  
    pass  
class TaskScreen(Screen):  
    pass  
class MyApp(App):  
    def build(self):  
        return Builder.load_string(kv)  
MyApp().run()
```

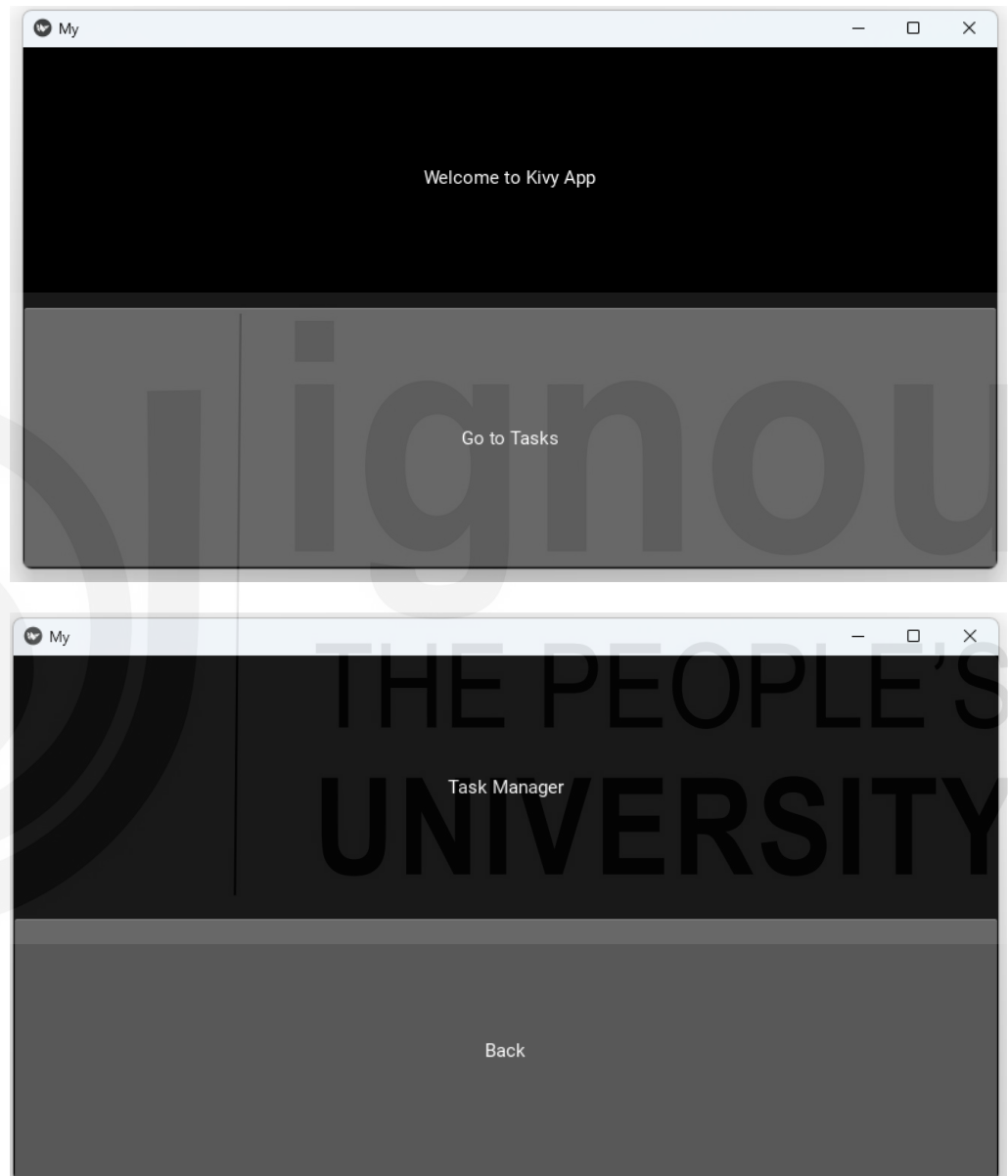


Figure 29: Output Screen for Organizing Apps with Multiple Screens

Styling with KV Language

You can add colors, font sizes, and spacing:

```
<TaskScreen>:  
    BoxLayout:  
        orientation: "vertical"  
        padding: 20  
        spacing: 10  
        Label:
```

```

text: "Task Manager"
font_size: "24sp"
color: 1,0,0,1
TextInput:
    id: task_input
    hint_text: "Enter a new task"
Button:
    text: "Add Task"
    on_press: app.add_task(task_input.text)

```

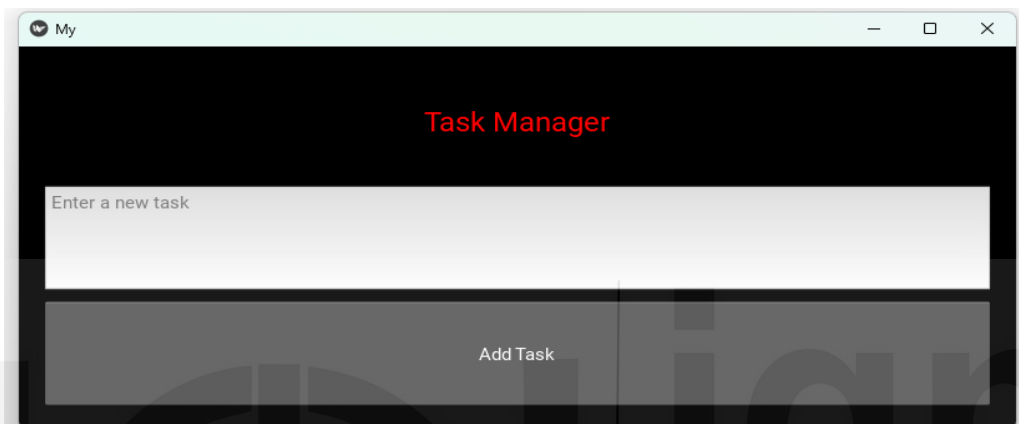


Figure 30: Output Screen for Styling with KV Language

Handling Input and Updating UI

```

class MyApp(App):
    tasks = []
    def build(self):
        return Builder.load_string(kv)
    def add_task(self, task):
        if task.strip():
            self.tasks.append(task.strip())
            print("Tasks:", self.tasks)

```

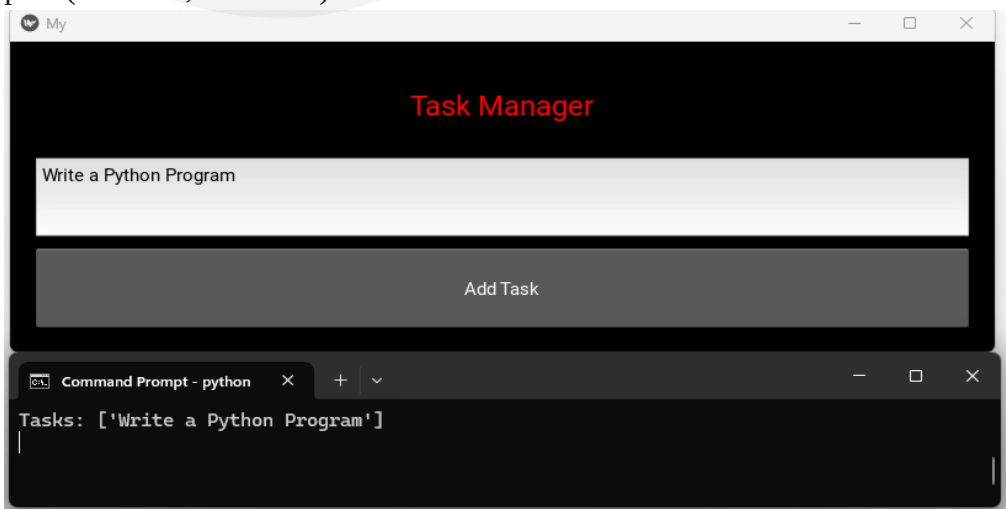


Figure 30: Output Screen for Handling Input and Updating UI

Saving & Loading Data with JSON

```
import json
class MyApp(App):
    tasks = []
    data_file = "tasks.json"
    def add_task(self, task):
        if task.strip():
            self.tasks.append(task.strip())
    def save_tasks(self):
        with open(self.data_file, "w") as f:
            json.dump(self.tasks, f)
    def on_start(self):
        try:
            with open(self.data_file) as f:
                self.tasks = json.load(f)
        except FileNotFoundError:
            self.tasks = []
```

For example, `tasks.json` will contain:

```
[
    "Buy groceries",
    "Complete Python assignment",
    "Prepare presentation"
]
```

Responsive Layouts

Kivy automatically adapts to window size, but you can use **relative layouts** for better responsiveness.

<TaskScreen>:

RelativeLayout:

Button:

text: "Bottom Right"

size_hint: 0.3, 0.2

pos_hint: {"right": 1, "y": 0}

Comparison of Python GUI Frameworks

Feature	Tkinter	PyQt	Kivy	wxPython
Included in Python	Yes	No (needs installation)	No	No
Modern UI	Basic look	Native and modern	Custom, touch-friendly	Native look

Feature	Tkinter	PyQt	Kivy	wxPython
Cross-Platform	Yes	Yes	Yes	Yes
Use Case Fit	Simple apps	Professional desktop apps	Mobile + Desktop apps	Native Windows/Linux apps

Check Your Progress -5

1. What is Kivy primarily used for?

- a) Backend development
- b) Web development
- c) Cross-platform mobile and desktop GUI development
- d) Database management

2. Which language does Kivy use for designing interfaces?

- a) XML
- b) HTML
- c) Python only
- d) KV language

3. What does the App class in Kivy do?

- a) Stores data
- b) Handles user input
- c) Manages the main application logic and window
- d) Creates animations

4. Which Kivy widget is used for single-line text input?

- a) Label
- b) Button
- c) TextInput
- d) Spinner

5. Which method is overridden to define the UI layout in a Kivy app?

- a) setup()
- b) run()
- c) build()
- d) layout()

6. What is Kivy and what makes it unique?

.....

7. Explain the role of the KV language in Kivy.

.....

8. What is a widget in Kivy?

.....

9. Differentiate between Label and TextInput.

10. What does the run() method do in a Kivy application?

11. Create a Task Manager with Search, Sorting & Status Update in Kivy with the following features:

- Add tasks with description + priority.
 - Mark tasks as *Pending*, *In Progress*, *Completed*.
 - Search dynamically.
 - Sort by **Priority** or **Status**.
 - Persistent storage in JSON.
 - Uses **RecycleView** for task listing (scalable for large lists).
-
-

9.7 SUMMARY

Python provides several GUI (Graphical User Interface) libraries that enable developers to build desktop applications featuring visual components like buttons, menus, input fields, and images. Each GUI toolkit offers unique features, making them suitable for various project requirements and levels of application complexity. In this unit, Tkinter, PyQt, wxPython, PySimpleGUI, and KivyGUI (Graphical User Interface) libraries are discussed with programming examples.

9.8 SOLUTION TO CHECK YOUR PROGRESS

Answer to Check Your Progress- 1

1. b) tkinter
2. d) mainloop()
3. a) Label
4. b) Executes a function on click
5. pack
6. Entry
7. Canvas
8. config

9.
A short code snippet to create a window with a title “Welcome”.

```
python
import tkinter as tk
root = tk.Tk()
root.title("Welcome")
root.mainloop()
```



Figure 31: Output Screen for create a window

10.
`pack()` – Arranges widgets vertically or horizontally in blocks (simplest layout).

- `grid()` – Arranges widgets in a tabular/grid (rows and columns) structure.
- `place()` – Allows placing widgets at exact coordinates (absolute positioning).

11.
Tkinter program to create an Entry box that accepts user input and displays it in a Label when a button is pressed.

```
import tkinter as tk
def display_input():
    user_text = entry.get()
    label.config(text="You entered: " + user_text)
root = tk.Tk()
root.title("Check Your Progress")
entry = tk.Entry(root, width=25)
entry.pack(pady=10)
button = tk.Button(root, text="Submit", command=display_input)
button.pack(pady=5)
label = tk.Label(root, text="", font=("Arial", 14))
label.pack(pady=10)
root.mainloop()
```

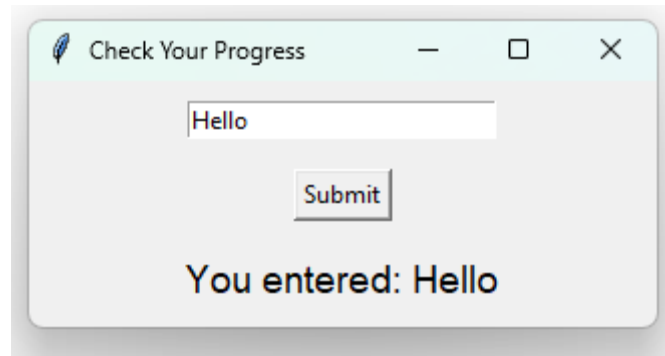


Figure 32: Output Screen for create an Entry box

12.

A Student Management System in Tkinter with the following features:

- a) Add Student → Enter Name, Roll No, and Marks
- b) View Students → Display all students in a table.
- c) Delete Student → Select and delete a student.
- d) Exit → Close the application.

```
import tkinter as tk
from tkinter import ttk, messagebox
students = [] # list to store student data
def add_student():
    name = entry_name.get()
    roll = entry_roll.get()
    marks = entry_marks.get()

    if name and roll and marks:
        students.append((name, roll, marks))
        update_table()
        entry_name.delete(0, tk.END)
        entry_roll.delete(0, tk.END)
        entry_marks.delete(0, tk.END)
    else:
        messagebox.showwarning("Input Error", "Please fill all fields!")
def update_table():
    for row in tree.get_children():
        tree.delete(row)
    for student in students:
        tree.insert("", tk.END, values=student)
def delete_student():
    selected = tree.selection()
    if selected:
        item = tree.item(selected)
        students.remove(item['values'])
        tree.delete(selected)
    else:
        messagebox.showwarning("Delete Error", "No student selected!")
root = tk.Tk()
root.title("Student Management System")
root.geometry("600x400")
```

```
# Input Frame
frame_input = tk.Frame(root)
frame_input.pack(pady=10)
tk.Label(frame_input, text="Name:").grid(row=0, column=0, padx=5, pady=5)
entry_name = tk.Entry(frame_input)
entry_name.grid(row=0, column=1, padx=5, pady=5)

tk.Label(frame_input, text="Roll No:").grid(row=0, column=2, padx=5,
pady=5)
entry_roll = tk.Entry(frame_input)
entry_roll.grid(row=0, column=3, padx=5, pady=5)

tk.Label(frame_input, text="Marks:").grid(row=0, column=4, padx=5, pady=5)
entry_marks = tk.Entry(frame_input)
entry_marks.grid(row=0, column=5, padx=5, pady=5)

tk.Button(frame_input, text="Add Student",
command=add_student).grid(row=0, column=6, padx=10)

# Table Frame
frame_table = tk.Frame(root)
frame_table.pack(pady=10)

columns = ("Name", "Roll No", "Marks")
tree = ttk.Treeview(frame_table, columns=columns, show="headings",
height=10)
for col in columns:
tree.heading(col, text=col)
tree.column(col, width=120)
tree.pack()

# Action Buttons
frame_buttons = tk.Frame(root)
frame_buttons.pack(pady=10)
tk.Button(frame_buttons, text="Delete Student",
command=delete_student).grid(row=0, column=0, padx=10)
tk.Button(frame_buttons, text="Exit", command=root.destroy).grid(row=0,
column=1, padx=10)
root.mainloop()
```

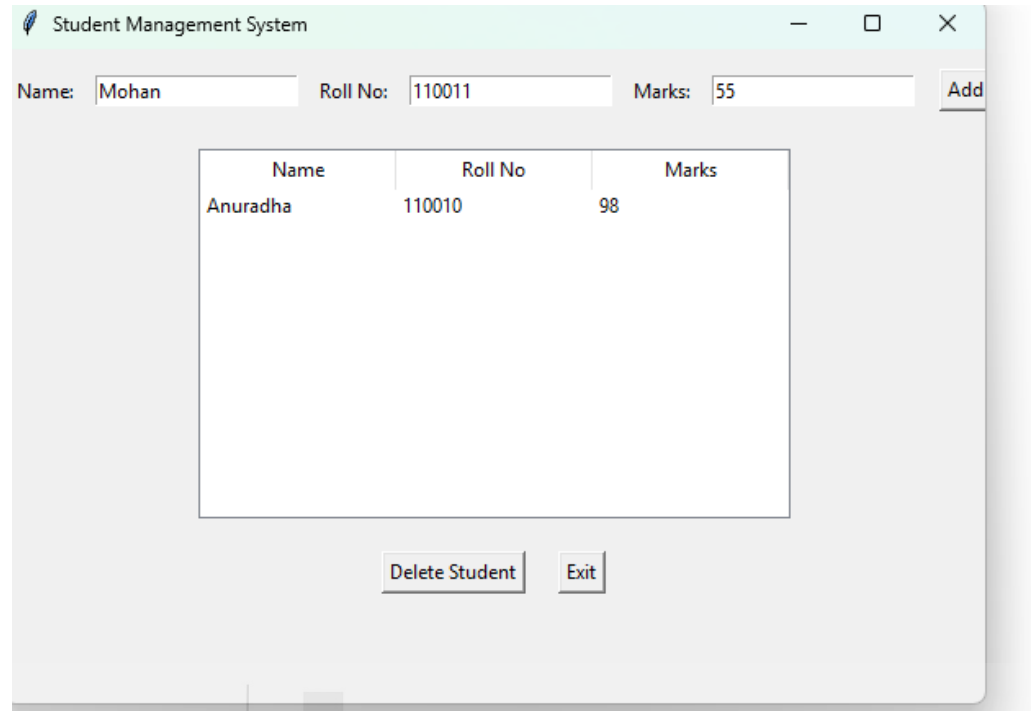


Figure 33: Output Screen for Student Management System

Answer to Check Your Progress-2

1. c) Initialize and manage the GUI application
2. b) QPushButton
3. c) show()
4. c) An event triggered by user interaction
5. d) pyuic5
6. Layout managers in PyQt are used to arrange widgets within a window. They handle the positioning and resizing of widgets automatically.
Two common types:
 - QVBoxLayout – Vertically stacks widgets.
 - QGridLayout – Arranges widgets in rows and columns.
7. Signals are emitted when a user interacts with a widget and Slots are methods/functions connected to signals that perform actions. Example:
`btn.clicked.connect(print_message)`
`def print_message():`
`print("Button was clicked")`
8. Qt Designer allows developers to create user interfaces visually using drag-and-drop. The .ui file generated can be converted to Python code using the pyuic5 command, saving time and reducing errors in manual coding.
9. PyQt Form with Name, Email, and Submit Button:

```
from PyQt6.QtWidgets import QApplication, QWidget, QLabel, QLineEdit,  
QPushButton, QVBoxLayout
```

```
def on_submit():  
    # Retrieve text from the input fields  
    name = name_input.text()  
    email = email_input.text()  
    print(f"EnterName: {name}")  
    print(f"EnterEmail: {email}")  
  
# Initialize the application  
app = QApplication([])  
  
# Create the main window  
window = QWidget()  
window.setWindowTitle("User Form")  
  
# Create a vertical layout  
layout = QVBoxLayout()  
  
# Add Name field  
layout.addWidget(QLabel("Name:"))  
name_input = QLineEdit()  
layout.addWidget(name_input)  
  
# Add Email field  
layout.addWidget(QLabel("Email:"))  
email_input = QLineEdit()  
layout.addWidget(email_input)  
  
# Add Submit button and connect it to the function  
submit_button = QPushButton("Submit")  
submit_button.clicked.connect(on_submit)  
layout.addWidget(submit_button)  
  
# Set layout, display window, and start the event loop  
window.setLayout(layout)  
window.show()  
app.exec()
```

ignou
THE PEOPLE'S
UNIVERSITY

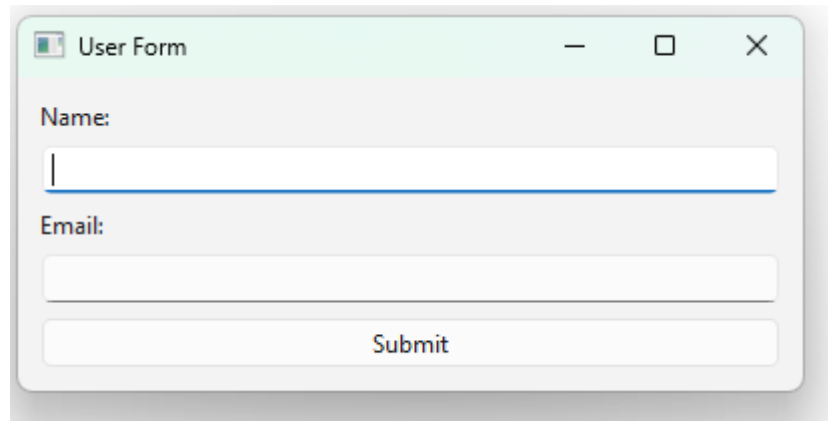


Figure 34: Output Screen for PyQt Form

Answer to Check Your Progress -3

- 1.c) wx.Frame
- 2.c) Initializing the window and entering the main loop
3. c) wx.StaticText
4. c) Displays a widget or window on screen
5. a) wx.BoxSizer
6. wx.TextCtrl: A text field for user input ,wx.StaticText: Displays read-only static text on the window.
7.
Sizers in wxPython are layout managers that automatically arrange and resize widgets according to window size and available space, ensuring responsive designs.
8.
A Task Manager with Search, Sorting & Status Update in wxPython with the following features:

```
import wx
import json
import os
```

```
DATA_FILE = "tasks_wx.json"
```

```
class TaskManager(wx.Frame):
    def __init__(self, parent=None, title="Advanced Task Manager - wxPython
Example"):
        super().__init__(parent, title=title, size=(750, 450))
```

```
self.tasks = []
self.load_tasks()
```

```
        panel = wx.Panel(self)
        vbox = wx.BoxSizer(wx.VERTICAL)
```

```
        # --- Input Section ---
        hbox1 = wx.BoxSizer(wx.HORIZONTAL)
```

```

self.task_input = wx.TextCtrl(panel)
self.priority_box = wx.ComboBox(panel, choices=["Low", "Medium",
"High"], style=wx.CB_READONLY)
self.priority_box.SetSelection(1) # Default Medium
add_btn = wx.Button(panel, label="Add Task")
add_btn.Bind(wx.EVT_BUTTON, self.on_add_task)
    hbox1.Add(self.task_input, proportion=2, flag=wx.ALL | wx.EXPAND,
border=5)
    hbox1.Add(self.priority_box, proportion=1, flag=wx.ALL | wx.EXPAND,
border=5)
    hbox1.Add(add_btn, proportion=0, flag=wx.ALL, border=5)
vbox.Add(hbox1, flag=wx.EXPAND)

# --- Search & Sort Section ---
hbox2 = wx.BoxSizer(wx.HORIZONTAL)
hbox2.Add(wx.StaticText(panel, label="Search:"), flag=wx.ALL |
wx.ALIGN_CENTER_VERTICAL, border=5)

self.search_input = wx.TextCtrl(panel)
self.search_input.Bind(wx.EVT_TEXT, self.refresh_table)

self.sort_box = wx.ComboBox(panel, choices=["Sort by Priority", "Sort by
Status"], style=wx.CB_READONLY)
self.sort_box.SetSelection(0)
self.sort_box.Bind(wx.EVT_COMBOBOX, self.refresh_table)

    hbox2.Add(self.search_input, proportion=2, flag=wx.ALL |
wx.EXPAND, border=5)
    hbox2.Add(self.sort_box, proportion=1, flag=wx.ALL | wx.EXPAND,
border=5)
vbox.Add(hbox2, flag=wx.EXPAND)

# --- Task Table ---
self.table = wx.ListCtrl(panel, style=wx.LC_REPORT |
wx.BORDER_SUNKEN)
self.table.InsertColumn(0, "Description", width=400)
self.table.InsertColumn(1, "Priority", width=100)
self.table.InsertColumn(2, "Status", width=120)
self.table.Bind(wx.EVT_LIST_ITEM_ACTIVATED, self.toggle_status)
vbox.Add(self.table, proportion=1, flag=wx.ALL | wx.EXPAND, border=5)

# --- Save Button ---
save_btn = wx.Button(panel, label="Save Tasks")
save_btn.Bind(wx.EVT_BUTTON, self.on_save_tasks)
vbox.Add(save_btn, flag=wx.ALL | wx.ALIGN_RIGHT, border=5)

panel.SetSizer(vbox)
self.refresh_table()

# -----
# Core Functionalities
# -----

def on_add_task(self, event):
    desc = self.task_input.GetValue().strip()

```

```
        if not desc:
wx.MessageBox("Task description cannot be empty.", "Input Error", wx.OK |
wx.ICON_WARNING)
        return
        priority = self.priority_box.GetValue()
        task = {"description": desc, "priority": priority, "status": "Pending"}
self.tasks.append(task)
self.task_input.Clear()
self.refresh_table()

def toggle_status(self, event):
    """Change status when row is double-clicked"""
    index = event.GetIndex()
    task = self.filtered_tasks[index]

    if task["status"] == "Pending":
        task["status"] = "In Progress"
    elif task["status"] == "In Progress":
        task["status"] = "Completed"
    else:
        task["status"] = "Pending"

self.refresh_table()

def refresh_table(self, event=None):
    """Update table with search & sorting"""
    search_text = self.search_input.GetValue().lower()
    self.filtered_tasks = [t for t in self.tasks if search_text in
t["description"].lower()]

    if self.sort_box.GetSelection() == 0: # by priority
        order = {"High": 0, "Medium": 1, "Low": 2}
    self.filtered_tasks.sort(key=lambda x: order[x["priority"]])
    else: # by status
        order = {"Pending": 0, "In Progress": 1, "Completed": 2}
    self.filtered_tasks.sort(key=lambda x: order[x["status"]])

self.table.DeleteAllItems()
    for task in self.filtered_tasks:
        index = self.table.InsertItem(self.table.GetItemCount(),
task["description"])
self.table.SetItem(index, 1, task["priority"])
self.table.SetItem(index, 2, task["status"])

def load_tasks(self):
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r") as f:
self.tasks = json.load(f)

def on_save_tasks(self, event):
    with open(DATA_FILE, "w") as f:
json.dump(self.tasks, f, indent=2)
```

```
wx.MessageBox("Tasks saved successfully!", "Saved", wx.OK |  
wx.ICON_INFORMATION)
```

```
if __name__ == "__main__":  
    app = wx.App(False)  
    frame = TaskManager()  
    frame.Show()  
    app.MainLoop()
```

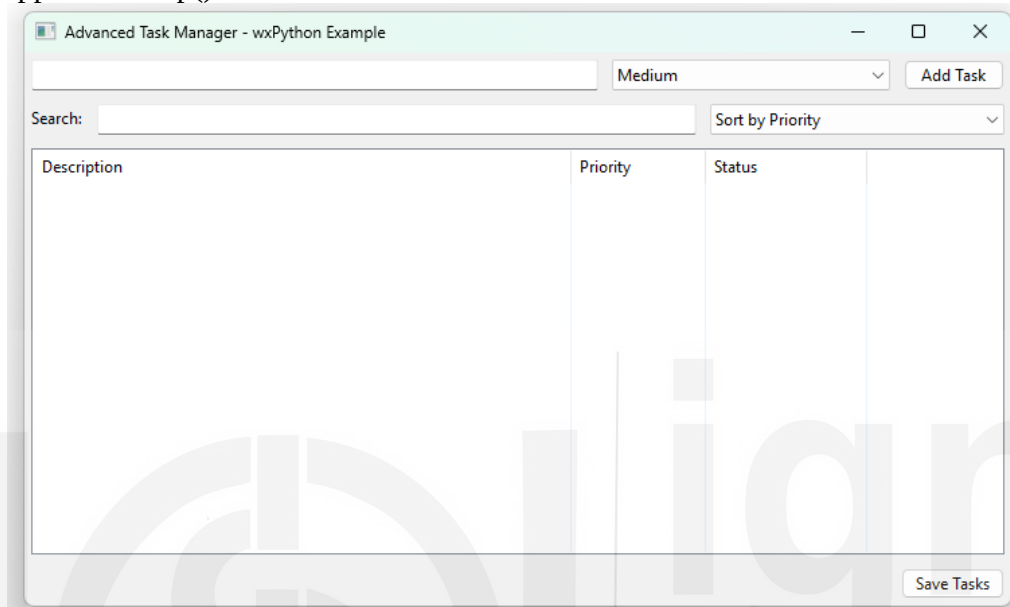


Figure 35: Output Screen for Task Manager with Search, Sorting & Status Update

Answer to Check Your Progress -4

- 1.c) Simplifying GUI development in Python
- 2.d) window.read()
3. c) Shows static text
4. c) window.close()

5.
PySimpleGUI is a Python library designed to simplify GUI programming. It provides an easy-to-use interface over other GUI toolkits like Tkinter, Qt, and WxPython.

6.
Use sg.Input() inside the layout and window.read() to capture the input.

7.
sg.Button(): Creates a clickable button for user actions and sg.Text(): Displays static text

8.
The event loop continuously reads user inputs and GUI events, enabling interaction with the application.

9.
Program written using FreeSimpleGUI:

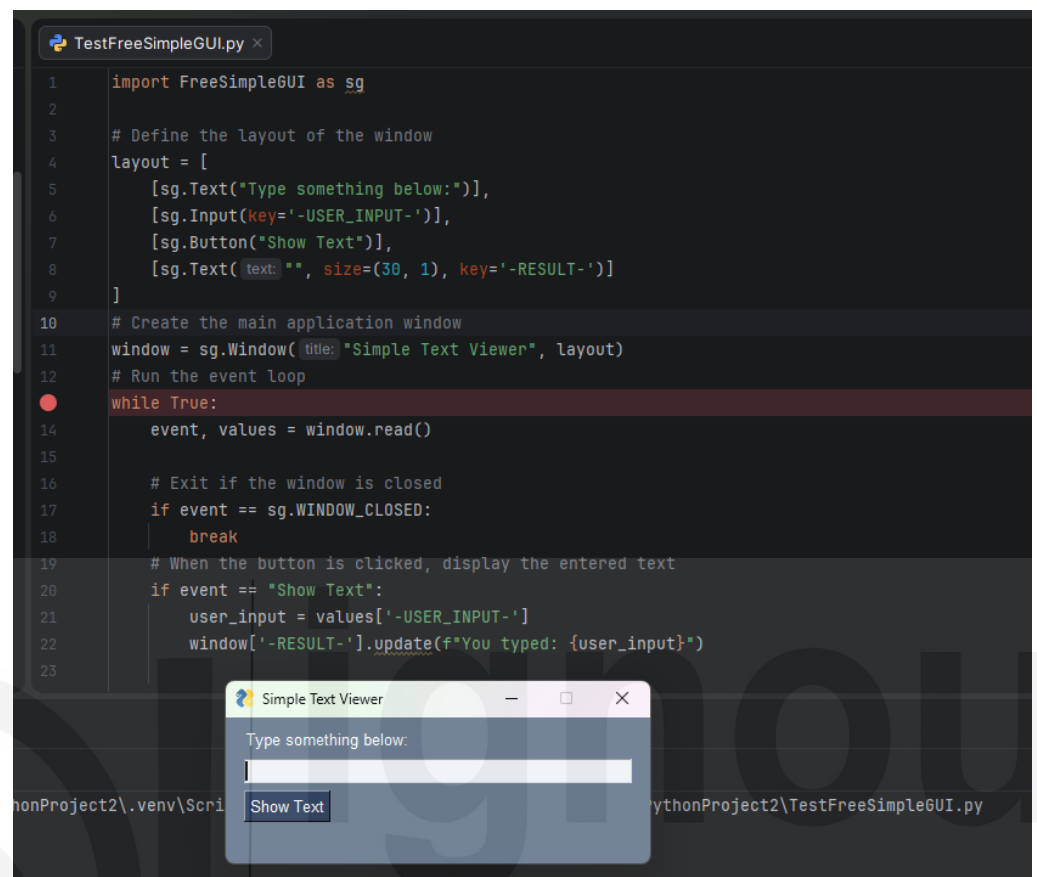


Figure 36: Output Screen for Simple Text Viewer

Answer to Check Your Progress -5

- 1.c) Cross-platform mobile and desktop GUI development
- 2.d) KV language
- 3.c) Manages the main application logic and window
- 4.c) TextInput
- 5.c) Build()

6.
Kivy is the Python open-source framework designed for creating multi-touch applications. It enables cross-platform development and uses the KV language to simplify and streamline user interface design.

7.
KV language is a special syntax used to separate the layout design from Python logic. It allows for more readable and maintainable code.

8.
A widget is a visual element in a GUI, such as a button, label, text box, or slider, that allows users to interact with the application

9.
Label: Used to display static text. TextInput: Used to allow user input.

10.
The run() method starts the application's main loop and displays the interface.

11.
import json
import os
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.recycleview import RecycleView
from kivy.uix.recycleview.views import RecycleDataViewBehavior
from kivy.uix.label import Label
from kivy.uix.behaviors import ButtonBehavior
from kivy.uix.textinput import TextInput
from kivy.uix.spinner import Spinner
from kivy.uix.button import Button
from kivy.uix.popup import Popup

```
DATA_FILE = "tasks_kivy.json"
class TaskButton(ButtonBehavior, Label):
    """Clickable label for task rows"""
    def __init__(self, manager, task, **kwargs):
        super().__init__(**kwargs)
        self.manager = manager
        self.task = task
        self.text = f"{task['description']} | {task['priority']} | {task['status']}"
        self.halign = "left"
        self.valign = "middle"
        self.bind(on_release=self.toggle_status)

    def toggle_status(self, instance):
        """Cycle status on click"""
        status = self.task["status"]
        if status == "Pending":
            self.task["status"] = "In Progress"
        elif status == "In Progress":
            self.task["status"] = "Completed"
        else:
            self.task["status"] = "Pending"
        self.manager.refresh_view()
```

```
class TaskRecycleView(RecycleView):
    def __init__(self, manager, **kwargs):
        super().__init__(**kwargs)
        self.manager = manager
        self.viewclass = TaskButton
        self.refresh_from_data()
```

```
    def update_data(self, tasks):
        self.data = [{"manager": self.manager, "task": t} for t in tasks]
        self.refresh_from_data()
```

```
class TaskManager(BoxLayout):
```

```
def __init__(self, **kwargs):
    super().__init__(orientation="vertical", **kwargs)
    self.tasks = []
    self.load_tasks()

    # --- Input Section ---
    input_layout = BoxLayout(size_hint_y=None, height=40)
    self.task_input = TextInput(hint_text="Enter task...", multiline=False)
    self.priority_spinner = Spinner(text="Medium", values=["Low", "Medium",
    "High"])
    add_btn = Button(text="Add Task")
    add_btn.bind(on_release=self.add_task)
    input_layout.add_widget(self.task_input)
    input_layout.add_widget(self.priority_spinner)
    input_layout.add_widget(add_btn)
    self.add_widget(input_layout)

    # --- Search & Sort Section ---
    control_layout = BoxLayout(size_hint_y=None, height=40)
    self.search_input = TextInput(hint_text="Search...", multiline=False)
    self.search_input.bind(text=self.on_search_change)
    self.sort_spinner = Spinner(text="Sort by Priority", values=["Sort by Priority",
    "Sort by Status"])
    self.sort_spinner.bind(text=self.on_sort_change)
    control_layout.add_widget(self.search_input)
    control_layout.add_widget(self.sort_spinner)
    self.add_widget(control_layout)

    # --- Task List ---
    self.rv = TaskRecyclerView(manager=self)
    self.add_widget(self.rv)

    # --- Save Button ---
    save_btn = Button(text="Save Tasks", size_hint_y=None, height=40)
    save_btn.bind(on_release=self.save_tasks)
    self.add_widget(save_btn)

    # Initial Load
    self.refresh_view()
    def load_tasks(self):
        if os.path.exists(DATA_FILE):
            with open(DATA_FILE, "r") as f:
                self.tasks = json.load(f)
    def save_tasks(self, *args):
        with open(DATA_FILE, "w") as f:
            json.dump(self.tasks, f, indent=2)
        popup = Popup(title="Saved", content=Label(text="Tasks saved
        successfully!"),
        size_hint=(0.5, 0.3))
        popup.open()
    def add_task(self, *args):
        desc = self.task_input.text.strip()
        if desc:
            self.tasks.append({"description": desc, "priority": self.priority_spinner.text,
            "status": "Pending"})
    self.task_input.text = ""
```

```

self.refresh_view()
    def on_search_change(self, instance, value):
self.refresh_view()
    def on_sort_change(self, instance, value):
self.refresh_view()
    def refresh_view(self):
search_text = self.search_input.text.lower()
    filtered = [t for t in self.tasks if search_text in t["description"].lower()]

    if self.sort_spinner.text == "Sort by Priority":
        order = {"High": 0, "Medium": 1, "Low": 2}
    filtered.sort(key=lambda x: order[x["priority"]])
    else:
        order = {"Pending": 0, "In Progress": 1, "Completed": 2}
    filtered.sort(key=lambda x: order[x["status"]])

self.rv.update_data(filtered)

class TaskApp(App):
    def build(self):
        return TaskManager()

if __name__ == "__main__":
    TaskApp().run()

```

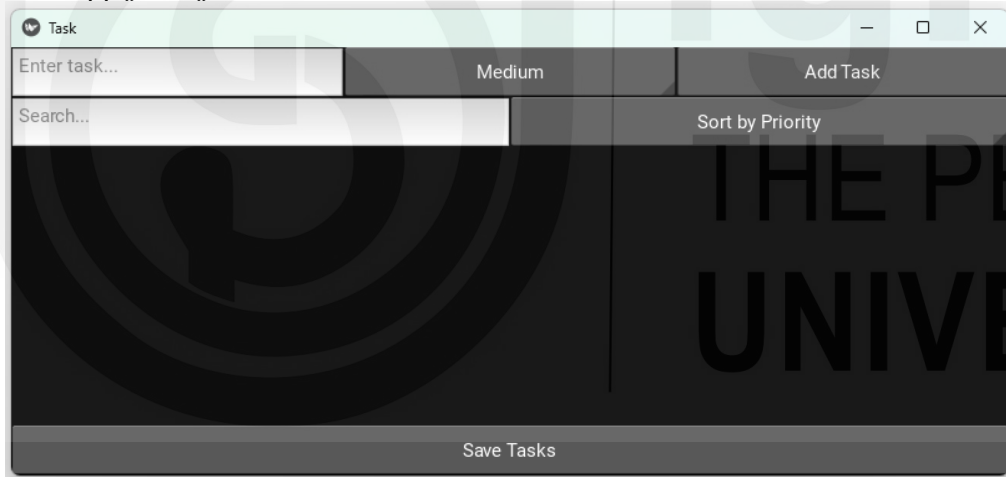


Figure 37: Output Screen for Task Manager

9.9 FURTHER READINGS

1. Chandrakar, S. and Bahadure, N. B. (2023). *Building Modern GUIs with Tkinter and Python*. BPB Publications, New Delhi.
2. Chandrakar, S. and Bahadure, N. B. (2023). *Python GUI with PyQt*. BPB Publications, New Delhi.
3. Rappin, N. and Dunn, R. (2006). *wxPython in Action*. Manning Publications, Greenwich, Connecticut.
4. PySimpleGUI. *PySimpleGU Cookbook* (<https://docs.pysimplegui.com/cookbook/>) and PySimpleGUI Documentation(<https://docs.pysimplegui.com/>) .
5. Ulloa, R. (2015). *Kivy – Interactive Applications and Games in Python*. Packt Publishing, Birmingham.